PAR2QO: Parametric Penalty-Aware Robust Query Optimization

Haibo Xiu, Yang Li, Qianyu Yang, Pankaj K. Agarwal, and Jun Yang
Duke University, Durham, NC, USA
{haibo.xiu,yang.li,qianyu.yang}@duke.edu,{pankaj,junyang}@cs.duke.edu

ABSTRACT

Parametric Query Optimization (PQO) is a critical challenge in database systems, yet existing approaches suffer from high training costs, sensitivity to estimation errors, and vulnerability to severe performance regressions. This paper introduces **PAR^{2}QO** (**Par**ametric **P**enalty-**A**ware **R**obust **Q**uery **O**ptimization), a system that integrates robust query optimization (RQO) into PQO. PAR^{2}QO strategically obtains plans from a well-balanced set of probe locations informed by the workload, and caches them as plan-penalty profiles. At runtime, PAR^{2}QO selects the plan with the lowest expected penalty, explicitly accounting for selectivity uncertainties. Extensive experiments across multiple benchmarks show that PAR^{2}QO delivers significant speedups over existing PQO methods while ensuring robustness against performance degradation. Additionally, we introduce **CARVER**, a workload generator guided by cardinality coverages of sub-queries, enhancing both the generality and stability of PQO models. We demonstrate CARVER can enhance the generality and stability of PQO methods, providing a reliable framework for evaluating and improving the optimization of parametric queries.

PVLDB Reference Format:
Haibo Xiu, Yang Li, Qianyu Yang, Pankaj K. Agarwal, and Jun Yang. PAR2QO: Parametric Penalty-Aware Robust Query Optimization. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Hap-Hugh/PAR2QO.

1 INTRODUCTION

A long-standing challenge in databases is *Robust Query Optimization* (*RQO*) [11]. Query optimizers rely on selectivity and cost estimates to pick execution plans. When these estimates are inaccurate, the chosen plan can perform far worse than the optimal. There are various approaches towards robust query optimization; we focus on finding a *robust query plan* in this work: instead of minimizing the plan cost given the estimated selectivities, which can be wrong, we look for a plan that "performs well" despite uncertainties in the estimates. In other words, when executed at the true selectivities, the plan is expected to incur a low penalty with respect to the cost of the true optimal plan. The resulting optimization problem is challenging because we must consider possible estimation errors and compare with the real optimal plans at the corrected selectivities, which amounts to exploring a neighborhood — rather than a single point — in the selectivity space. The optimization overhead can thus be higher than that for traditional query optimization.

Parametric Query Optimization (*PQO*) is a technique originally motivated by the desire to reduce optimization overhead for queries that follow the same template but differ in their parameter settings, which frequently arise in applications. Once PQO finds the optimal plan for a given query instance, it caches the plan and reuses it for future query instances whose estimated selectivities are close to the cached plan. Pushing the idea further, Kepler [14], the latest state of the art in PQO, employs deep learning to learn a function that selects the right plan among a set of cached candidates.

Combining PQO and RQO is natural, because PQO helps to amortize query optimization overhead across multiple query instances. The potential of this combination was demonstrated in the recent RQO work of PARQO [47], which considered caching one robust plan per query template. However, opportunities for such reuses are limited because the choice of the robust plan varies across the selectivity space. We believe that there are better ways to reuse the computation in RQO. Intuitively, RQO explores the neighborhood of each query instance; even if two query instances are not close enough to justify the same robust plan, the work of exploring their neighborhoods can still be reused.

Existing PQO approaches, on the other hand, are not designed to find robust plans. For some approaches, components of their solutions employ heuristics that favor caching plans that tend to be more robust, e.g., plans that are close to being optimal at multiple points in the selectivity space. However, these approaches do not consider robustness when selecting plans at runtime. Furthermore, their ability of caching a robust plan depends heavily on what candidate plans were considered for caching in the first place. As a result, their candidate plans are either too focused on training workload queries (e.g., [41]) without accounting for errors in selectivity estimates, too spread across the entire selectivity space (e.g., [12]), and/or very expensive to acquire (e.g., Kepler [14], or [12] in high dimensions).

To overcome these limitations, we propose a new PQO system, *PAR^{2}QO* (***PAR**ameteric **P**enalty-**A**ware **R**obust **Q**uery **O**ptimization*). PAR^{2}QO offers a systematic treatment of robustness, guided by a formal definition of robustness against errors in selectivity estimates. It is practical to deploy, requiring only non-intrusive access to existing query optimizers and reasonable training and setup time. It also makes its decisions in a transparent manner, with no "black-boxing," which is easy to understand and debug. Specifically, we make the following contributions in this paper:

- By focusing its learning efforts on selectivity errors observed from training queries, PAR^{2}QO uses only a tiny fraction of the training cost of Kepler. PAR^{2}QO's learned error model informs generation of relevant candidate plans at preparation time and

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

plan selection at runtime, and is able to achieve better query performance than Kepler despite shorter training time.

- PAR^{2}QO employs a novel sampling method to acquire candidate plans. Leveraging the training workload and the learned error model, PAR^{2}QO is able to focus on the most relevant regions of the selectivity space — not only points corresponding to the estimated selectivities of workload queries, but also neighborhoods induced by uncertainty in their estimates.
- PAR^{2}QO uses a novel caching strategy in the form of *plan-penalty profiles*. Instead of caching only robust plans, this data structure caches candidate plan profiles that allow the best robust plan to be computed at runtime, consistent with the robustness objective. Instead of relying on a black box, the computation is explainable using the error model and the plan profiles.
- In the process of training and evaluating our approach and others, we observed that standard benchmarks such as JOB [30] and DSB [10] are lacking in the PQO setting, because they either offer no guidance on how to generate parameter settings for a query template, or generate them from simple distributions that fail to stress-test PQO. Therefore, we propose a new query workload generator, *CARVER* (***CAR**dinality co**VER**age*), which ensures that its generated queries provide good coverage of the range of possible subquery cardinalities that might be encountered by the optimizer. When used for training workloads, CARVER also improves the robustness of PQO and its ability to generalize to new workloads.

Overall, by defining the robust optimization objective clearly and designing methods accordingly, PAR^{2}QO is able to guard against query performance degradations due to selectivity errors and deliver better overall query performance than previous approaches. Experiments show that PAR^{2}QO outperforms other methods across various benchmarks and workloads, achieving up to 1.96× speedup over PostgreSQL and 1.83× over Kepler on JOB, while avoiding severe regressions observed in other approaches (Sections 5.1 and 5.2). PAR^{2}QO also demonstrates robustness across shifting data distributions (Section 5.3) and different workloads (Section 5.4).

2 PRELIMINARIES AND BACKGROUND

2.1 Problem Statements and Assumptions

Assumptions and Prerequisites. As discussed in Section 6, there are many different approaches to robust query optimization, e.g., from worst-case optimal join algorithms to runtime adaptive query processing, but this paper targets query optimizers that generate traditional (non-adaptive) query plans based on cardinality estimates. Our solution is designed to work on existing database systems without changing their plan space, execution engine, or optimization procedure. We only assume that the system supports (or can be easily extended to support) the following calls. First, given a query Q, $\widehat{\mathsf{Sel}}(Q)$ returns a vector of selectivity estimates for "relevant" subqueries of Q (we will elaborate on what "relevant" means later). Second, $\mathsf{Opt}(Q, \mathbf{s})$ returns the optimal execution plan for query Q given the vector $\mathbf{s}$ of relevant selectivities. Third, $\mathsf{Cost}(\pi, \mathbf{s})$ returns the estimated cost of execution plan π given the vector $\mathbf{s}$ of relevant selectivities. Typically, the cost of optimization is dominated by Opt; individual $\widehat{\mathsf{Sel}}$ and Cost calls are far cheaper in comparison [16]. For our PostgreSQL-based implementation, we use `EXPLAIN` to obtain estimates of costs and relevant selectivities, the same mechanism in [20, 47] for selectivity injection, and `pg_hint_plan` [35] for specifying the plan.

Parametric Query Optimization (PQO). A query template Γ has a set $\mathsf{params}(\Gamma)$ of query parameters drawn from various domains. A query (instance) Q following Γ, denoted by $Q \sim \Gamma$, assigns a specific set $\mathsf{params}(Q)$ of values to the set of parameters in $\mathsf{params}(\Gamma)$. To avoid repeating expensive Opt calls for workloads with many instances of the same query template, PQO precomputes and/or maintains information for Γ that enables it to efficiently find, for a given query $Q \sim \Gamma$, an execution plan whose cost is not much higher than that of $\mathsf{Opt}(Q, \widehat{\mathsf{Sel}}(Q))$, without making an Opt call.

Typically, PQO uses a per-template *plan cache*, which contains a set of *candidate plans* for the template, and selects the best plan from them for each incoming query at runtime.

Robust Query Optimization (RQO). In reality, when optimizing a query Q, the system does not have the true selectivities $\mathbf{s}$, but instead their estimates $\hat{\mathbf{s}}$. As a result, it may select a plan whose execution cost is much higher than that of the true optimal plan. To quantify the penalty of making a wrong selection and to define a "robust" plan formally, we adopt the following general framework of PARQO [47] based on stochastic optimization, which is flexible enough to encode many notions of robustness found in previous work [1, 3, 45] and in applications.

We assume a user-defined *penalty function* $\mathsf{Penalty}(\pi, \mathbf{s})$ specifying the penalty incurred by executing a potentially non-optimal plan π under the true selectivities $\mathbf{s}$. We use the same default as in [47], but note that our approach works for any choice of $\mathsf{Penalty}$:

$$\mathsf{Penalty}(\pi, \mathbf{s}) = \begin{cases} 0 & \text{if } \mathsf{Cost}(\pi, \mathbf{s}) \leq (1+\tau) \cdot \mathsf{Cost}^\star(\mathbf{s}), \\ \mathsf{Cost}(\pi, \mathbf{s}) - \mathsf{Cost}^\star(\mathbf{s}) & \text{otherwise.} \end{cases} \quad (1)$$

Here, $\mathsf{Cost}^\star(\mathbf{s}) = \mathsf{Cost}(\mathsf{Opt}(Q, \mathbf{s}), \mathbf{s})$ is the (surrogate) optimal cost, approximated by the estimated cost of the best plan obtained by the optimizer assuming the knowledge of the true selectivities $\mathbf{s}$; $\mathsf{Cost}(\pi, \mathbf{s})$ is the estimated cost of π when executing under $\mathbf{s}$. Equation (1) defines penalty to be proportional to the amount of cost exceeding the optimal, but only if it is beyond the prescribed tolerance ($\tau = 0.2$ following the convention in [1, 9, 12, 45, 47]).

Since selectivities are uncertain, we model them as a random vector $\mathbf{S}$. Assuming that $\mathbf{S} \sim f(\mathbf{s}|\hat{\mathbf{s}})$, the probability density function for the distribution of true selectivities $\mathbf{s}$ conditioned on the estimates $\hat{\mathbf{s}}$, we can then define the *most robust* plan for Q as the one that minimizes the expected penalty $\mathsf{E}[\mathsf{Penalty}(\pi, \mathbf{S}) \mid \widehat{\mathsf{Sel}}(Q)]$.

Problem Statements. The *parametric robust query optimization problem* is defined as follows. Given a query template Γ, a (training) database instance $\mathcal{D}$, and a (training) query workload W consisting of queries following Γ:

- (Cache preparation) Compute a data structure from $\mathcal{D}$ and W.
- (Runtime plan selection) Given a query $Q \sim \Gamma$, use the cached data structure to find a robust plan for Q.

We first present a baseline solution in Section 2.2 and then discuss our main solution in Section 3.

In a "cold-start" variant of the above setup, the training query workload W is not given. This variant additionally requires addressing the following problem: given a query template Γ and a database

instance $\mathcal{D}$, generate query workload W following Γ. Informally, we would like the generated workload to serve as a good workload for training and/or testing a parametric robust query optimizer. To this end, the generated queries should provide good coverage of the various selectivity scenarios that the optimizer can potentially encounter, including situations when the data distribution may change and the query distribution may be different. We present our solution in Section 4.

2.2 PARQO′: A PARQO-Based Baseline

As a baseline solution, we extend PARQO [47], a state-of-the-art robust query optimization framework. We first briefly review the components of PARQO, especially since its error profiling component is also used by our main solution in Section 3. Then we describe the extension of PARQO into a full-fledged parametric robust query optimization solution, which we call PARQO′ to differentiate from the version described in PARQO.

Error Profiling. PARQO uses the database instance $\mathcal{D}$ and the query workload W to learn an error model $\mathcal{E}$ for query template Γ that allows us to construct the conditional distribution of true selectivities $f(\mathbf{s}|\hat{\mathbf{s}})$ given the estimated selectivities $\hat{\mathbf{s}} = \widehat{\mathfrak{Sel}}(Q)$ for a query $Q \sim \Gamma$. In general, optimizing a complex query may involve many selectivities, from single-table selections to multi-table selection-join subqueries; it would be infeasible to acquire all possible selectivities and represent the full joint distribution of all their errors. Therefore, PARQO chooses to model $\mathcal{E}$ using a collection of *error profiles*, each capturing a *querylet*, i.e., a small subquery template (up to a three-table selection-join) in Γ. We collect these profiles by comparing the optimizer estimates with the real subqueries cardinalities observed by executing W over $\mathcal{D}$. Each error profile contributes a factor to the joint distribution $f(\mathbf{s}|\hat{\mathbf{s}})$. Further details are in [47].

Finding a Robust Plan. Given a plan $Q \sim \Gamma$ and $\hat{\mathbf{s}} = \widehat{\mathfrak{Sel}}(Q)$, PARQO computes a robust plan for Q per Equation (1) as follows. First, to tame the complexity of subsequent optimization, PARQO identifies a small number of *sensitive dimensions* among those of $\hat{\mathbf{s}}$ whose uncertainty impacts the penalty the most. This dimensionality reduction is guided by a careful sensitivity analysis that assesses each dimension's contribution to the overall variance in penalty, using Monte-Carlo sampling from $f(\mathbf{s}|\hat{\mathbf{s}})$. Second, PARQO draws a number of selectivity samples from $f(\mathbf{s}|\hat{\mathbf{s}})$ restricted to the sensitive dimensions, and invokes Opt with sampled selectivities to obtain a set of candidate plans. Finally, PARQO returns the candidate plan with the least expected penalty, estimated by recosting the plan at sampled selectivities and comparing its costs with the respective optimal plans at these locations.

Parametric Robust Query Optimization using PARQO. The method above is designed to find a robust plan for a single query instance. Since the overhead of finding a robust plan is high, PARQO proposes a simple method that caches a robust plan per query template. Let Q_c denote the query that the cached robust plan was optimized for, and let $\hat{\mathbf{s}}_c$ denote the estimated selectivities for Q_c. At runtime, given a new query Q with the same template and estimated selectivities $\hat{\mathbf{s}}$, PARQO tests if the Kullback–Leibler (KL) divergence of $f(\mathbf{s}|\hat{\mathbf{s}})$ from $f(\mathbf{s}|\hat{\mathbf{s}}_c)$ is below a prescribed threshold θ_{KL}. If yes, Q and Q_c will be almost indistinguishable from the perspective of a cardinality-based optimizer, so PARQO simply returns the cached plan for Q. Otherwise, PARQO falls back to calling $\mathsf{Opt}(Q, \hat{\mathbf{s}})$ to obtain a plan for Q, with no guarantee of robustness.

We extend the above method to obtain a baseline solution for the parametric setting as follows. To prepare the cache:

- Learn an error model $\mathcal{E}$ from $\mathcal{D}$ and W.
- Construct $\mathcal{C} = \{\langle \pi_i, \mathbf{s}_i \rangle\}$, which consists of a set of candidate robust plans (π_i) along with the respective selectivities ($\mathbf{s}_i$) they were obtained under. Starting with an empty $\mathcal{C}$, we iterate through each query $Q \in W$ with estimated selectivities $\hat{\mathbf{s}}$. We calculate the KL divergence of $f(\mathbf{s}|\hat{\mathbf{s}})$ from $f(\mathbf{s}|\mathbf{s}_i)$ for each candidate $\langle \pi_i, \mathbf{s}_i \rangle \in \mathcal{C}$. If the divergence is below θ_{KL} for at least one existing candidate, meaning that Q is already "covered" by $\mathcal{C}$, we move on. Otherwise, we use PARQO to find a robust plan π for Q and add $\langle \pi, \hat{\mathbf{s}} \rangle$ to $\mathcal{C}$.

At runtime, given a query Q, we find the candidate $\langle \pi_i, \mathbf{s}_i \rangle \in \mathcal{C}$ that minimizes the KL divergence of $f(\mathbf{s}|\widehat{\mathfrak{Sel}}(Q))$ from $f(\mathbf{s}|\hat{\mathbf{s}}_i)$, and return π_i as a robust plan for Q. No Opt or Cost calls are needed.

3 PARAMETRIC PENALTY-AWARE ROBUST QUERY OPTIMIZATION

A main issue with PARQO′ in Section 2.2 is that in order for a query Q to reuse a cached plan π_i reliably, $f(\mathbf{s}|\widehat{\mathfrak{Sel}}(Q))$ must be very similar to $f(\mathbf{s}|\mathbf{s}_i)$, so many $\langle \pi_i, \mathbf{s}_i \rangle$ may be needed to cover the entire selectivity space. However, finding each robust plan to add to the cache is expensive, which makes the cache preparation cost high. A key insight is to reuse the computation involved in finding a robust plan, rather than reusing the result plans directly. We illustrate the opportunities for reusing computation with a simple conceptual example. For brevity, we will refer to $\widehat{\mathfrak{Sel}}(Q)$ as the "location" of query Q in the selectivity space. Intuitively, $f(\mathbf{s}|\widehat{\mathfrak{Sel}}(Q))$ defines a "neighborhood" around (but not necessarily centered at) Q, and finding a robust plan for Q involves exploring this neighborhood. Suppose PARQO′ has cached two plans π_1 and π_2 whose neighborhoods are both near Q's, but neither is close enough for Q to justify reuse. Nonetheless, the two neighborhoods together provide sufficient coverage for Q's neighborhood. Recall that finding π_1 and π_2 in the first place required sampling plans in their neighborhoods. The same samples should be useful for finding the robust plan for Q.

This insight leads us to our new solution PAR^{2}QO. It changes the strategy of caching robust plans for previously seen queries of a template to caching a data structure $\mathfrak{P}$ called the *plan-penalty profiles*. On a high level, $\mathfrak{P}$ compactly represents how a set of robust plan candidates performs relative to the optimal plans (per Equation (1)) over a set of *probe locations* in the selectivity space. PAR^{2}QO intelligently samples probe locations and uses the optimal plans to seed the set of candidate plans; it then reduces this set to construct a compact $\mathfrak{P}$. At runtime, given a query Q, PAR^{2}QO uses $\mathfrak{P}$ to estimate the expected penalty of candidate plans over $f(\mathbf{s}|\widehat{\mathfrak{Sel}}(Q))$, by reweighing the per-location penalties accordingly to remove sampling bias, and selects the best candidate plan.

While the high-level idea is simple, interesting challenges lie in how to design an effective sampling scheme for probe locations and how to reduce the candidate plan set in order to facilitate the

identification of robust plans at runtime. In the remainder of this section, we describe the details of PAR^{2}QO. Cache preparation proceeds in three steps:

- Learn an error model $\mathcal{E}$ from $\mathcal{D}$ and W. This step is exactly the same as PARQO′ discussed in Section 2.2.
- Sample a set $\mathcal{L}$ of probe locations (Section 3.1).
- Compute the plan-penalty profiles $\mathfrak{P}$ (Section 3.2).

Selection of robust plan at runtime is covered in Section 3.3. Finally, Section 3.4 presents an in-depth discussion of PAR^{2}QO by contrasting it with other state-of-the-art approaches, highlighting the novelty of our solution and how it represents a more principled approach to achieving plan robustness, whose advantages will also be empirically demonstrated in Section 5.

3.1 Sampling Probe Locations

There are two natural ideas for sampling probe locations. First, sampling should be informed by the training query workload W because, intuitively, we do not want to devote much effort to regions of the selectivity space that do not matter to queries. Second, for each query Q, we want not only to probe the location $\widehat{\mathsf{Sel}}(Q)$, but also to sample locations in the neighborhood defined by $f(\mathbf{s}|\widehat{\mathsf{Sel}}(Q))$, because these locations might yield the true optimal plan for Q.

A naive approach is to randomly sample some locations in the neighborhood of each workload query. However, there are two pitfalls with this approach. First, if the workload W is skewed, particular regions of the selectivity space will be over-sampled. Such high concentrations of samples are wasteful because they have diminishing returns on improving our understanding of these regions. Second, unbalanced sample densities can be problematic for downstream tasks, as more weights are put on popular queries at the expense of less common ones, even if the latter may be costly and stand to benefit more from robust plans.

Our idea is to employ a check akin to the reuse check in PARQO′ (Section 2.2) to prevent over-sampling in a region: if a workload query "hits" a region that we have already sampled, we do not sample more from this region. We also track how often a region is hit by workload queries, which empirically captures the query distribution and allows us to estimate the sampling probabilities of probe locations, useful for bias correction later (Section 3.3).

The algorithm proceeds as follows. We organize the set of sampled probe locations into clusters $\{\langle \mathcal{L}_i, \mathbf{s}_i, h_i \rangle\}$, where $\mathcal{L}_i$ denotes the set of probe locations in cluster i, $\mathbf{s}_i$ serves to remember the distribution $f(\mathbf{s}|\mathbf{s}_i)$ from which $\mathcal{L}_i$ was sampled, and h_i tracks how many queries in W hits this cluster. For each workload query $Q \in W$, we find the cluster $\langle \mathcal{L}_i, \mathbf{s}_i, h_i \rangle$ that minimizes the KL divergence of $f(\mathbf{s}|\widehat{\mathsf{Sel}}(Q))$ from $f(\mathbf{s}|\mathbf{s}_i)$. If this minimum divergence is below the prescribed threshold θ_{KL}, we increment h_i by one and move on. Otherwise, we sample n locations from $f(\mathbf{s}|\widehat{\mathsf{Sel}}(Q))$, and create a new cluster $\{\langle \mathcal{L}_{\mathsf{new}}, \widehat{\mathsf{Sel}}(Q), 1 \rangle\}$, where $\mathcal{L}_{\mathsf{new}}$ contains the newly sampled locations. At the end of the process, let $m \leq |W|$ denote the final number of clusters. Overall, $\mathcal{L}$ is the union of all $\mathcal{L}_i$'s, with mn probe locations total, to compute plan-penalty profiles.

3.2 Computing Plan-Penalty Profiles

Conceptually, the plan-penalty profiles $\mathfrak{P}$ is a matrix whose rows correspond to candidate plans and columns correspond to the probe locations in $\mathcal{L}$. The entry $\mathfrak{P}[\pi, \mathbf{s}]$ records the value of $\mathsf{Penalty}(\pi, \mathbf{s})$ as defined in Equation (1), and the row vector $\mathfrak{P}[\pi, \cdot]$ represents the penalty profile for plan π.

As the first step in building $\mathfrak{P}$, we obtain a set $\mathcal{P}$ of candidate plans and construct a *plan-cost matrix* $\mathbf{C}$, where each row vector $\mathbf{C}[\pi, \cdot]$ represents the *cost profile* of candidate plan $\pi \in \mathcal{P}$, i.e., its costs at all probe locations. To collect the set $\mathcal{P}$ of candidate plans, we call $\mathsf{Opt}(Q, \mathbf{s})$ to obtain the optimal plan for each probe location $\mathbf{s} \in \mathcal{L}$. Note that $|\mathcal{P}| \leq |\mathcal{L}|$ because it is possible for the optimizer to choose the same plan for multiple locations. Then, for each distinct candidate $\pi \in \mathcal{P}$, we call $\mathsf{Cost}(\pi, \mathbf{s})$ for each $\mathbf{s} \in \mathcal{L}$, and store the result in entry $\mathbf{C}[\pi, \mathbf{s}]$. The overall running time of this step is dominated by $|\mathcal{L}|$ calls to Opt and $|\mathcal{P}| \cdot |\mathcal{L}|$ calls to Cost.

At this point, we can simply compute $\mathfrak{P}$ from $\mathbf{C}$ by calculating each entry $\mathfrak{P}[\pi, \cdot]$ using Equation (1), with $\mathsf{Cost}(\pi, \mathbf{s}) = \mathbf{C}[\pi, \mathbf{s}]$ and $\mathsf{Cost}^\star(\mathbf{s}) = \min_{\pi' \in \mathcal{P}} \mathbf{C}[\pi', \mathbf{s}]$. However, the size of $\mathfrak{P}$ can be large because of the number of candidate plans. For example, in our experiments in Section 5, templates 22 and 24 from the JOB benchmark [30] produced more than 300 candidate plans. Hence, we next focus on reducing the set $\mathcal{P}$ of candidate plans to a small number k. After reduction, the size of $\mathfrak{P}$ becomes $\mathrm{O}(k|\mathcal{L}|)$. Besides saving space and improving inference speed, a good candidate plan reduction method can also help improve robustness (and hence query performance), a point that we will further validate empirically in Section 5. We consider two plan reduction methods below.

Reduction by τ-Approximate Cover. The first method follows a natural heuristic: we prefer to keep candidate plans that are near-optimal at a large number of probe locations. The same heuristic and its variants have been employed by various PQO methods including QueryLog [41] and Kepler [14]. More precisely, we say that a plan π *covers* probe location $\mathbf{s}$ if $\mathsf{Cost}(\pi, \mathbf{s}) \leq (1 + \tau) \cdot \mathsf{Cost}^\star(\mathbf{s})$, where τ is the same as in Equation (1). Given k, the target number of candidate plans after reduction, we want to choose a subset of k plans in $\mathcal{P}$ that together cover as many probe locations as possible.

This problem is an instance of the set cover problem [44], where a simple greedy algorithm offers an approximation ratio of $H(|\mathcal{L}|) \leq \ln |\mathcal{L}| + 1$ (in terms of the number of candidates needed to cover all probe locations). In each step, we greedily pick the candidate plan that covers the most number of probe locations that have not been covered yet. The running time is $\mathrm{O}(|\mathcal{P}| \cdot |\mathcal{L}|)$, with no optimizer calls, and is negligible compared with the time to acquire $\mathbf{C}$.

We expect this method to work well in practice, because its heuristic generally reflects robustness. However, it is possible to miss the robust plans for certain locations. The reason is that the heuristic, by considering the overall number of probe locations covered, reflects a "global" measure of robustness, while the selection of robust plans in fact depends on a more "local" neighborhood around the query of interest.

Reduction by Cost Profile Similarity. Our second method follows a more conservative approach to choose "representative" candidate plans that can generally substitute for others, without heuristically favoring those that are globally more robust; later, the runtime plan selection procedure will pick, among these representatives, the best plan for a new query according to the more precise robustness criterion. To this end, we measure the distance between two candidate plans π and π' using the Jensen–Shannon distance between the two

discrete distributions P and P' represented by their cost profiles $\mathbf{C}[\pi, \cdot]$ and $\mathbf{C}[\pi', \cdot]$. More precisely, this distance is calculated as the square root of the average between the KL divergence from P to M and the KL divergence from P' to M, where $M = (P + P')/2$ is a mixture distribution of P and P'. Given k, the target number of candidate plans after reduction, we want to choose a subset of k plans in $\mathcal{P}$ that minimizes the maximum distance between any plan in $\mathcal{P}$ to its closest among the k chosen.

This problem is an instance of the metric k-center problem [44]. A simple greedy algorithm based on the farthest-first traversal gives an approximation ratio of 2. We choose the first candidate plan randomly from $\mathcal{P}$ and place it in our solution set S. While $|\mathcal{S}| < k$, we add to S the plan π in $\mathcal{P} \setminus S$ that maximizes the distance between π and plans already in S. We stop and return S once $|\mathcal{S}| = k$. The running time is $\mathrm{O}(k \cdot |\mathcal{P}| \cdot |\mathcal{L}|)$, where the factor $\mathrm{O}(|\mathcal{L}|)$ reflects the running time of one distance computation. Again, no optimizer calls are required, so the time is negligible compared with the time to acquire $\mathbf{C}$. An example comparing the plans reduced by both methods is provided in the extended version of this paper[48].

3.3 Runtime Plan Selection

At runtime, PAR^{2}QO has access to the error model $\mathcal{E}$, the plan-penalty profiles $\mathfrak{P}$ with k candidate plans (Section 3.2), as well as additional information on the probe locations $\mathcal{L}$ (Section 3.1). Given a query Q, PAR^{2}QO uses $\mathcal{E}$ to derive $f(\mathbf{s}|\widehat{\mathrm{Sel}}(Q))$, the distribution of true selectivities conditioned on the estimates. Then, PAR^{2}QO estimates the expected penalty (Section 2.1) for each of the k candidate plans, and returns the plan with the lowest estimate. To estimate the expected penalty $\mathsf{E}[\mathsf{Penalty}(\pi, \mathbf{S}) \mid \widehat{\mathrm{Sel}}(Q)]$ for candidate plan π, we use its penalty profile $\mathfrak{P}[\pi, \cdot]$. If the probe locations were sampled from $f(\mathbf{s}|\widehat{\mathrm{Sel}}(Q))$, the average penalty would yield an unbiased estimate of the expected penalty. However, since the probe locations were sampled differently, we must reweigh the penalties to correct the bias using the method of importance sampling.

Recall from Section 3.1 that the probe locations are organized into m clusters with n locations in each. For the i-th cluster $\langle \mathcal{L}_i, \mathbf{s}_i, h_i \rangle$, its n probe locations $\mathcal{L}_i = \{\mathbf{s}_{i,1}, \mathbf{s}_{i,2}, \ldots, \mathbf{s}_{i,n}\}$ were sampled from $f(\mathbf{s}|\mathbf{s}_i)$. Furthermore, since the i-th cluster was hit h_i times, we estimate the sampling probability of the cluster to be $h_i / \sum_{i=1}^{m} h_i = h_i/|W|$. Combining the two observations above, we estimate the sampling probability of location $s_{i,j}$ as $(h_i/|W|) \cdot f(\mathbf{s}|\mathbf{s}_i)$. Hence, overall, the expected penalty of π is estimated as:

$$\frac{1}{mn} \sum_{i=1}^{m} \sum_{j=1}^{n} \frac{f(\mathbf{s}_{i,j}|\widehat{\mathrm{Sel}}(Q))}{(h_i/|W|) \cdot f(\mathbf{s}_{i,j}|\mathbf{s}_i)} \cdot \mathfrak{P}[\pi, s_{i,j}].$$

Since we use expected penalties to rank plans, we can simplify the above calculation into the following, by dropping terms that do not affect ranking:

$$\sum_{i=1}^{m} \sum_{j=1}^{n} \frac{f(\mathbf{s}_{i,j}|\widehat{\mathrm{Sel}}(Q))}{h_i f(\mathbf{s}_{i,j}|\mathbf{s}_i)} \cdot \mathfrak{P}[\pi, s_{i,j}].$$

As an additional optimization, we can also cache the value of $f(\mathbf{s}_{i,j}|\mathbf{s}_i)$ when generating $\mathbf{s}_{i,j}$. Overall, the selection of the robust plan can be done by a single pass over $\mathfrak{P}$, so the running time is $\mathrm{O}(k|\mathcal{L}|)$. No optimizer calls are needed except the single $\widehat{\mathrm{Sel}}(Q)$ call to obtain the estimated selectivities.

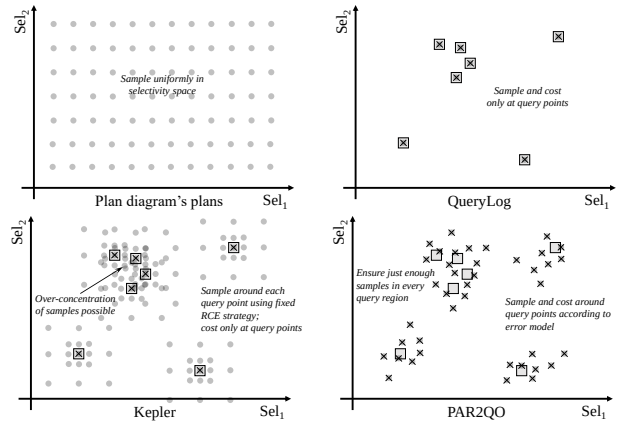


Figure 1: Illustration of locations in a 2-d selectivity space where various PQO approaches obtain candidate plans (circles) and cost them (crosses), using training queries (squares) when applicable.

3.4 Discussion

We contrast PAR^{2}QO with other PQO approaches, including two state-of-the-art systems, Kepler [14] and QueryLog [41]. Before delving deeper, we note that previous PQO approaches do not specifically optimize for finding *robust* plans. In fact, traditional PQO would be happy with reproducing the plans selected by underlying optimizers, with no consideration for plan robustness. However, it would be incorrect to conclude that traditional PQO cannot *beat* the underlying optimizer by definition. Once we closely examine many of the PQO methods, we see that many have components whose heuristics favor keeping robust plans as candidates, thereby allowing them to beat the underlying optimizer when it makes incorrect selectivity estimates, even if this feature is not by design.

We first highlight differences between PAR^{2}QO and Kepler [14].

(1) Kepler executes plans for all training queries as well as alternative plans obtained via *row-count evolution* (RCE). Measuring actual execution times makes Kepler robust against errors not only in selectivity estimation but also in plan cost models. In contrast, PAR^{2}QO only executes (counting versions of) training queries (and no alternative plans) to learn an error model for selectivities, and it "trusts" $\mathsf{Cost}(\pi, \mathbf{s})$ given correct $\mathbf{s}$. Hence, PAR^{2}QO spends significantly less timepreparing its cache than Kepler. In Section 5, we will see that this focus on selectivity errors is capable of delivering comparable or better query performance than Kepler despite a much lower initial cost.

(2) Kepler explores the "neighborhood" of a training query Q using RCE, which perturbs subquery selectivities in order to discover more candidate plans. This method hence improves robustness, analogous to PAR^{2}QO sampling from $f(\mathbf{s}|\widehat{\mathrm{Sel}}(Q))$ according to the error model (Section 3.1). However, RCE uses a fixed strategy for generating perturbations (exponentially spaced in selectivity ranges), which we believe is not as effective as PAR^{2}QO's perturbations informed by the error model.

(3) The same τ-approximate cover method in Section 3.2 is employed by Kepler to reduce the set of candidate plans. As discussed earlier, this method helps to improve "global" robustness (over the training workload). However, unlike PAR^{2}QO,

which has a built-in mechanism (Section 3.1) to avoid over-representing popular queries, the effectiveness of Kepler's plan reduction is susceptible to an imbalanced query workload. Moreover, in determining coverage, Kepler does not consider the costs of candidate plans at locations obtained by RCE; hence, its notion of coverage is much narrower than ours.

(4) Kepler makes its runtime plan selection using a neural network model, combined with a method for assessing prediction uncertainty. However, it is difficult to justify plan choices and interpret uncertainty. In contrast, PAR^{2}QO's runtime plan selection is simple and transparent; the availability of the error model and the plan penalty profiles makes the selection process easy to understand.

Next, we compare PAR^{2}QO with PQO approaches that are not based on deep learning. The most recent representative is Querylog [41], but we will also touch on some earlier approaches.

(1) To find candidate plans, previous work has generally used one of the following two methods. The first method, used by earlier approaches including the pioneering work of *anorexic plan diagrams* [12], attempts to cover the entire selectivity space, but quickly becomes intractable beyond 5-6 dimensions. The second method, used by QueryLog, is to sample past queries for candidate plans. However, Querylog has no mechanism for further perturbing selectivities to obtain potentially better plans, like PAR^{2}QO's sampling (Section 3.1) or Kepler's RCE.

(2) To reduce the set of candidate plans, most PQO approaches, like Kepler, use the τ-approximate cover method in Section 3.2 or some variants. For example, anorexic plan diagrams use the same method; Querylog seeks a cover that minimizes an "aggregate sub-optimality" measure, which is close in spirit to this method. While this reduction naturally favors robust plans, its benefit is limited here because of how the candidates were generated in the first place: neither uniformly sampling the entire selectivity space nor focusing on the locations of the training queries alone comes close to PAR^{2}QO in the ability to capture the effect of selectivity estimation errors on robustness.

(3) To select among candidate plans at runtime, QueryLog employs a model using contextual multi-armed bandit learning, which can adapt dynamically over time, while earlier approaches generally perform no better than recosting all candidate plans for the query and picking the cheapest. None aims at picking a robust plan like PAR^{2}QO does.

In summary, PAR^{2}QO offers a systematic and coherent treatment of robustness — driven by a formal definition of robustness against errors in selectivity estimates — from the initial learning of the error model, sampling of probe locations and candidate plans, construction of the plan-penalty profiles, to the final plan selection at runtime. Furthermore, PAR^{2}QO makes decisions transparently, with no "black-boxing," which are easy to interpret and debug.

4 CARVER: WORKLOAD GENERATION GUIDED BY CARDINALITY

Training and testing query optimizers require a query workload, whose choice is not always clear. Synthetic benchmarks (e.g., DSB [10]) typically use standard distributions (e.g., Gaussian) to generate parameter settings for given templates. Alternatively, if the database has been operating for sometime, past queries can be used for the workload. In either case, there is no guarantee that such workloads will capture future queries. If our goal is *robust* query optimization, it is natural to make training and testing queries cover the possible scenarios that a query optimizer may potentially encounter, as comprehensively as possible. Indeed, recent work on Kepler [14] generates parameter settings for a given query template by uniformly sampling rows from the full join result (with no selections applied), ensuring that every join result row has a chance to be covered by the generated query workload.

As argued in Section 1, however, we need a still better notion of coverage. Recall that a cost-based optimizer makes its decisions by estimating the selectivities of query predicates. Hence, it would be useful for the workload to include queries that exhibit a diverse range of selectivities. By randomly sampling final join result rows, Kepler will tend to overrepresent popular column values in its query parameters. Second, optimizer decisions are heavily influenced by the intermediate result set cardinalities of subqueries. Therefore, to better train and test the optimizer, we should also generate scenarios that cover the range of possible subquery cardinalities, instead of focusing only on the final join result as Kepler does.

CARVER meets the above requirements by providing a flexible and powerful way for users to ensure that the generated queries provide good coverage of the optimization scenarios. Formally, given a database instance $\mathcal{D}$ and a query template Γ with a set $\mathsf{params}(\Gamma)$ of parameters, our goal is to generate a set of queries following the template Γ, where each query assigns a specific value to each parameter in $\mathsf{params}(\Gamma)$. Overall, CARVER works as follows. It randomly generates queries following template Γ from a mixture distribution $\{(P_k, w_k)\}$, whose components (P_k's) and associated weights (w_k's) are specified by the user. Each component distribution P_k is specified by a set of subquery templates $\{\gamma_{k,j}\}$ of Γ, such that their parameters form a disjoint partitioning of $\mathsf{params}(\Gamma)$. P_k is defined as the product distribution $\prod_j P_{k,j}$, where each $P_{k,j}$ is a distribution governing the generation of values for $\mathsf{params}(\gamma_{k,j})$ in the subquery template $\{\gamma_{k,j}\}$.

Example 1. *example of a two-component mixture, where one is assembled by small querylets, while the other is just the overall query.*

We now describe the procedure for generating from P_{kj} for each $\mathsf{params}(\gamma_{k,j})$. We assume that $\gamma_{k,j}$ is a selection-join query template, and that it can be rewritten as $\sigma_\theta(q)$, where q is the (parameterless) *base query* obtained from $\gamma_{k,j}$ by "removing" all atomic query predicates involving parameters,[1] and θ is the *remainder predicate* involving $\mathsf{params}(\gamma_{k,j})$. The *selectivity* of a setting in $\mathsf{params}(\gamma_{k,j})$ is defined as $|\sigma_\theta(q(\mathcal{D}))|/|q(\mathcal{D})|$ where θ is instantiated with the given setting. Users can specify a bucketization B of the selectivity range (not necessarily equally spaced), e.g., $B = \{[0, 0.5\%), [0.5\%, 1\%), [1\%, 2\%), \ldots, [90\%, 1]\}$; CARVER will generate an equal number of $\mathsf{params}(\gamma_{k,j})$ settings for each bucket of B, up to the number of possible settings for this bucket.

Intuitively, including a subquery in the definition of mixture component guides CARVER to ensure that its generated parameters provide good coverage of the possible subquery cardinalities.

[1]The general procedure needs to handle subtleties that arise because of OR and NOT, which we omit here.

Additionally, including subqueries joining multiple tables and involving multiple query parameters ensures that CARVER accounts for dependencies among join and selection predicates.

Depending on the type and complexity of the remainder predicate θ, the generation procedure varies. We describe several common cases in Section 4.1 and leave the rest of the details to [48]. Section 4.2 concludes the section and discusses how CARVER handles some additional challenges.

4.1 Generation of Subquery Parameters

Equality Selections. Here, θ has the form $A_1 = v_1 \wedge \cdots \wedge A_n = v_n$, where each A_i is a column of some input table in q. First, we group the result rows of q by $A_1, \ldots, A_n$ and compute the row count for each group. The selectivity of θ for each group (using the group-by values for parameter setting) is simply the group row count divided by $|q|$, the total of all group row counts. Second, we sort the groups by their counts and bucketize them according to B. Finally, we draw an equal number of distinct samples from each bucket (unless limited by the number of groups available in the bucket). Note that the first and second steps can be computed conveniently in SQL using a `SELECT-FROM-WHERE-GROUPBY-ORDERBY` statement.

Note that the above procedure also handles the case of a single "not-equal-to" selection, i.e., θ has the form $A \neq v$, by computing the selectivity of v as one minus its group row count divided by $|q|$.

Inequality Selections. Suppose that θ has the form $A_1 \leq v_1 \wedge \cdots \wedge A_n \leq v_n$. As with the case of equality selections, we first group the result rows of q by $A_1, \ldots, A_n$ and compute the row count for each group. We then sort the groups by $A_1, \ldots, A_n$; the result is conceptually a n-dimensional array $\mathbf{A}$ representing a discrete probability density function. Let $\mathbf{A}[v_1, \ldots, v_n]$ denotes the array entry for $(v_1, \ldots, v_n)$. Next, we compute the cumulative density function (CDF) over $\mathbf{A}$ in-place (details to be presented shortly). We then sort $\mathbf{A}$ by the CDF values and bucketize them according to B. Finally, we draw an equal number of distinct samples from each bucket (again, subject to availability). All processing before the CDF computation can be done easily in SQL.

In more detail, the CDF can be computed as follows, by accumulating $\mathbf{A}$ over one additional dimension at a time. In the first step, for each setting of $A_1, \ldots, A_{n-1}$ to $v_1, \ldots, v_{n-1}$, we update the subarray $\mathbf{A}[v_1, \ldots, v_{n-1}, *]$ to be the conditional CDF over A_n by computing the prefix sum over its elements. In general, the step accumulating over A_i updates each subarray $\mathbf{A}[v_1, \ldots, v_{i-1}, *, \ldots]$ to be the conditional CDF over $A_i, \ldots, A_n$ while setting $A_1, \ldots, A_{i-1}$ to $v_1, \ldots, v_{i-1}$; each such subarray is updated in-place as the prefix sum over its constituent subarrays updated in the previous step. The last step updates $\mathbf{A}$ to be the overall CDF.

The above procedure can be easily extended to support other inequality predicates: $A_i < v_i$ can be supported by handling the bucket boundaries slightly differently; $A_i > v_i$ and $A_i \geq v_i$ can be handled by reversing A_i (from ascending to descending) in the sort order of the groups. It is also possible to support range predicates in the form of A_i `BETWEEN` l_i `AND` u_i by conceptually duplicating the column A_i in input data as columns A_i^l and A_i^u, and handling the predicate as $A_i^l \geq l_i \wedge A_i^u \leq u_i$.

`LIKE` *Selection.* Suppose that θ has the form A `LIKE` v. The design space for the generation procedure is large. We describe the token-based approach currently used by CARVER below; exploration of alternative approaches is left for future work. First, we make one pass over the result rows of q to tokenize all A values. If A in general contains multi-word strings, we tokenize each A value into words. Otherwise, we tokenize each A value into characters. During the pass, we maintain the set of unique tokens along with the number of q result rows whose A value contains this token. Second, we sort the tokens by their counts and bucketize them according to B. Finally, the same sampling procedure is carried out over the buckets. Each token *token* sampled corresponds to the predicate A `LIKE '%`*token*`%'`.

The above procedure can be extended to handle a conjunction of `LIKE` selections. For example, to handle A_1 `LIKE` $v_1 \wedge A_2$ `LIKE` v_2, for each result row q, we will tokenize both A_1 and A_2 values and consider all pairs of tokens, one from A_1 and the other from A_2. We maintain the set of unique token pairs along with the number of q result rows whose A_1 and A_2 contain the tokens in the pair, respectively.

4.2 Discussion

There are several points worth noting about CARVER. First, we emphasize that CARVER by design goes beyond merely covering ranges of selectivities for individual, single-table selections. The generation procedures in Section 4.1 generally support multi-table join subqueries and, when applicable, joint sampling of multiple query parameters. This feature ensures that the workload reflects the dependencies that exist among join and selection predicates, as handling such dependencies has been widely recognized [27, 29, 30] as more challenging for query optimizers than single-table selections selectivities.

Second, we note that in some cases, there are methods for combining the procedures in Section 4.1 to jointly generate parameters for more complex predicates. For example, it is not difficult to handle a conjunction of equality selections and inequality selections, or a conjunction of equality selections and `LIKE` selections. We will not exhaustively list all such possibilities here. Moreover, there always exists a default method for handling a combination of multiple parameterized predicates, simply by generating parameters independently using subquery templates that share the same base query but differ in their parameterized remainder predicates. This method can be applied when the complex predicate does not fall into a case discussed earlier, or when we prefer a cheaper method than the earlier ones.

Third, the base query q may be too expensive to compute if q joins many large tables and has few selection predicates (recall that all parameterized predicates go into the remainder predicate θ instead). In this case, we instead compute a sample of q's result rows. CARVER currently implements the simple approach of taking a uniform random sample of each table involved in the join. While this approach is widely known to suffer from low join rate, it has worked sufficiently well for experiments in Section 5. If needed, more sophisticated methods for sampling from join can be used instead [22, 26].

Finally, it is sometimes useful to require that every generated query returns a non-empty result set, particularly for production workloads where queries typically yield results, unlike ad hoc exploration workloads. There are also other technical reasons (see Section 5 for details). Therefore, CARVER offers an option to enforce non-empty results. When query parameters are sampled independently from multiple distributions, CARVER executes the query to verify validity and rejects settings that yield empty results. If the rejection rate is too high, CARVER suggests switching to a mixture component that samples directly from the full join result, which automatically ensures non-empty results by design.

5 EXPERIMENTS

Databases and benchmarks. We use two widely adopted benchmarks in our evaluation. **JOB** (Join Order Benchmark)[32] is based on real-world IMDB data consists of 33 query templates. It is designed to challenge query optimizers with complex characteristics, including skewed/correlated data distributions and intricate join relationships. **DSB**[10] is a recently introduced industrial benchmark built on synthetic data, extending TPC-DS[36] with more sophisticated data distributions and correlations. We concentrate on the 15 SPJ query templates, and use a scale factor of 2 for data generation.

Workload synthesis. For JOB, we generate three different workloads: CARVER, Uniform, and Historical. First, for **CARVER**, we consider the one-component scenario by default, regarding all 2-table querylets as the set of subquery templates $\{\gamma_{k,j}\}$ in our generator, and the selectivity bucketing for each $\mathsf{params}(\gamma_{k,j})$ is performed by partitioning the `group by` results into ten equally spaced selectivity ranges, i.e., $B = \{[0, 10\%), [10\%, 20\%), \ldots, [90\%, 1]\}$. For the **Uniform** workload, we first perform a full-table join across all tables based on the join conditions specified in the query templates. $\mathsf{params}(\Gamma)$ are then uniformly sampled from the resulting join output, same as the synthetic method in Kepler [14]. For **Historical** workload, we follow the methodology in PARQO [47]. Additional workload details are available in [48]. For DSB, in addition to CARVER, we generate a **Gaussian** workload by defining a Gaussian distribution centered at 0 with a standard deviation of 2. We then use `dsqgen`[10] to independently sample params for each binding parameter from this distribution. For each workload generation method, a posteriors filter is applied after initial synthesis to ensure only *valid* queries—those returning non-empty results—are included. We synthesize 250 unique queries for each $\mathsf{params}(\Gamma)$: 50 queries are used to initialize and train PQO models; the remaining 200 queries serve as the testing workload for evaluation.

PAR^{2}QO setup. We implement PAR^{2}QO on top of PostgreSQL V16.2, leveraging plan and selectivity injection tools from [20, 35, 47]. For each Γ, we learn an error model comprising $f(\mathbf{s}|\hat{\mathbf{s}})$ for each relevant querylets (up to 3-table subqueries). Each $f(\mathbf{s}|\hat{\mathbf{s}})$ is generated using kernel density estimators based on observed data from the training workload, following setups in [47]. Unless otherwise specified, we set n=50 as the selectivity probe size per cluster and $\theta_{\mathrm{KL}} = \ln(200)$ as the prescribed threshold. The sample size is user-configurable, and we demonstrate that even with this relatively small number of probe locations, PAR^{2}QO achieves strong performance. We set the target number k of candidate plans as $\max(0.2 \cdot |\mathcal{P}|, 10)$, where the minimum value of 10 ensures sufficient diversity when the initial $|\mathcal{P}|$ is small. τ-approximate cover (τ-Cover in short) is regarded as the default method for plan reduction, and the impact of different methods is analyzed in Section 5.2.

The remainder of this section is structured as follows: First, we compare PAR^{2}QO's performance against other PQO methods and baselines (Section 5.1). Next, Section 5.2 analyzes and demonstrates the contribution of each component in PAR^{2}QO, using stronger variants. Then, we assess the robustness under data distribution shifts in Section 5.3. Finally, Section 5.4 evaluates how CARVER outperforms other synthetic workloads in training PQO models.

5.1 Performance Evaluation of PQO Methods

We evaluate the performance of PAR^{2}QO by comparing it with the following PQO methods and baselines:

- **PostgreSQL**: A baseline method that invokes the optimizer separately for each query instance, relying on PostgreSQL's default behavior. While this serves as a foundational comparison, our goal extends beyond matching the optimizer's performance—we aim to enhance robustness and efficiency for parametric queries. Additional stronger baselines are introduced in Section 5.2.
- **Kepler**: Kepler[14] is a state-of-the-art PQO framework that learns from real query executions to predict the fastest plan. In experiments, we perturb all sub-queries to populate the initial candidate set, prune candidates using its default greedy algorithm, with a near-latency-optimal threshold of 1.2×, and train a model for each template.
- **PARQO′**: An extension of [47], as introduced in Section 2.2. In PARQO′, we employ the same error profiles and expected-penalty-based metric for robust plan selection as in PAR^{2}QO.
- **QueryLog′:** Following the plan population method in [41], this baseline method constructs the initial candidate set from optimal plans observed in the training workload, rather than selectivity probes as in PAR^{2}QO. It then applies τ-Cover plan reduction, and at runtime, selects the plan with the lowest recost, $\mathsf{Cost}(\pi, \widehat{\mathsf{Sel}}(Q))$.

Additional configuration details for Kepler and PARQO′are available in [48]. For a fair comparison, all PQO methods use the same set of training queries. To minimize system noise and maintain a warm-cache environment, each selected plan is executed 5 times and we report the median latency as the final result. All experiments were conducted on a Linux server equipped with 16 Intel(R) Core(TM) i9-11900 @ 2.50GHz processors.

PAR^{2}QO outperforms other methods in terms of average execution latency. Table 1 summarizes the results on JOB. PAR^{2}QO delivers the highest improvement on CARVER, achieving a 1.96× speedup over PostgreSQL and 1.83× over Kepler. While Kepler shows strong speedups on Uniform and Historical, it achieves only a 1.07× speedup on CARVER. On Historical, both Kepler and PAR^{2}QO reach approximately a speedup of 1.95× over PostgreSQL, higher than other methods. PARQO′ also demonstrates competitive performance but does not surpass PAR^{2}QO. Meanwhile, QueryLog′ shows regressions across all workloads compared to PostgreSQL, with the largest observed regression on CARVER at −1.10×.

Beyond average latency, our results show that PAR^{2}QO achieves a superior balance between efficiency and robustness. To evaluate

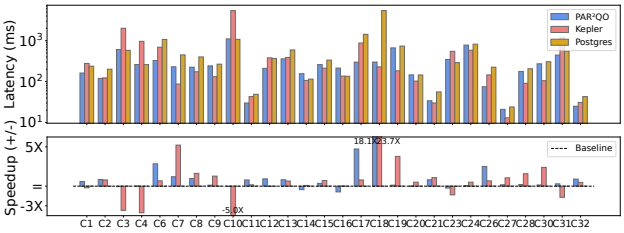
Table 1: Comparison of average execution latency (ms) and significant speedup (↑) / regression (↓) counts by different PQO methods, baselines and their variants across three workloads (CARVER, Uniform, and Historical) over 33 templates of JOB.

	CARVER				Uniform				Historical			
	Latency (ms)	↑ > 1.2×	↓ > 1.2×	↓ > 2×	Latency (ms)	↑ > 1.2×	↓ > 1.2×	↓ > 2×	Latency (ms)	↑ > 1.2×	↓ > 1.2×	↓ > 2×
PostgreSQL	536 ms	-	-	-	410 ms	-	-	-	637 ms	-	-	-
PAR^{2}QO	**273 (1.96×)**	15	3	0	**254 (1.61×)**	14	2	0	**326 (1.95×)**	15	7	0
Kepler	499 (1.07×)	**19**	5	4	306 (1.34×)	**15**	5	2	328 (1.94×)	**19**	2	1
PARQO′	305 (1.76×)	14	7	0	305 (1.34×)	13	5	0	348 (1.83×)	16	5	0
QueryLog′	587 (-1.10×)	4	6	0	428 (-1.04×)	5	4	1	668 (-1.05×)	2	7	0
Other Variants:												
QueryLog′(sample)	658 (-1.23×)	3	10	3	426 (-1.04×)	4	9	5	725 (-1.14×)	3	9	3
BestCost	344 (1.56×)	12	3	0	355 (1.15×)	12	3	0	400 (1.59×)	12	1	0
BestCost (bound)	513 (1.04×)	10	2	0	362 (1.13×)	11	3	0	591 (1.08×)	9	1	0
NearestSel	401 (1.34×)	13	9	2	422 (-1.03×)	9	6	1	377 (1.69×)	15	6	1
Ablation Study:												
PAR^{2}QO (similarity)	287 (1.87×)	14	4	0	270 (1.52×)	14	1	0	330 (1.93×)	17	5	0
PAR^{2}QO (w/o)	285 (1.88×)	14	4	0	255 (1.61×)	14	2	0	339 (1.88×)	13	8	1
BestCost (similarity)	420 (1.28×)	11	9	1	402 (1.02×)	9	2	1	446 (1.43×)	8	3	0
BestCost (w/o)	366 (1.46×)	8	3	0	390 (1.05×)	9	3	0	444 (1.43×)	7	4	0

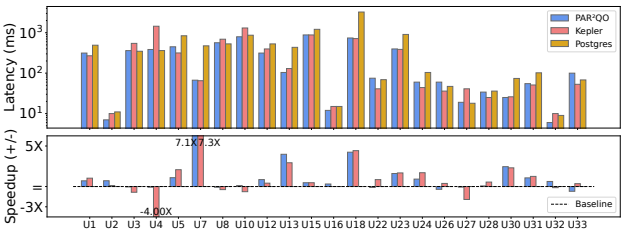
this, we analyze the number of query templates with *significant speedup* or *regression*. We define *significant* as a speedup or regression exceeding 1.2× and classify *severe* regressions as those worse than −2×. While significant speedups highlight a method's ability to identify efficient plans, robustness is equally critical in preventing significant and severe regressions. We observe that Kepler consistently achieves the largest number of speedup templates across all workloads, outperforming PAR^{2}QO. However, it is "fragile", frequently encountering severe regressions, where PAR^{2}QO successfully avoids such extreme cases. These poor-performing cases negatively impact Kepler's overall average latency.

To further analyze the performance at template-level, we focus on PAR^{2}QO and Kepler, presenting detailed execution latency and speedup results in Figure 2. For clarity and simplicity, each template is labeled with the workload's initial letter plus its ID (e.g., $C2$ represents template 2 in CARVER, and $H1$ is template 1 in Historical workload). We exclude templates in Figure 2 where both PAR^{2}QO and Kepler do not exhibit significant speedup or regression. Results show that Overall, Kepler can achieve markedly higher speedups in certain scenarios. For instance, on $C7$ and $C19$, it attains 5.17× and 4.03× speedups, respectively, whereas PAR^{2}QO only reaches 1.96× and 1.10×. Moreover, there are templates where Kepler provides a speedup but PAR^{2}QO does not, such as $H4$, $H8$, $H19$, $U22$, and $U26$. However, Figure 2 also highlights that PAR^{2}QO effectively mitigates severe regressions: the worst performance degradation for PAR^{2}QO occurs in $H4$ (−1.81×), while Kepler's largest regression, observed in $C10$, exceeds −5×. These findings indicate that Kepler, by learning from real executions, can sometimes identify *faster* plans. However, these plans lack robustness and fail to provide consistent efficiency in practice. In contrast, PAR^{2}QO effectively mitigates extreme performance degradations, leading to better overall execution latency.

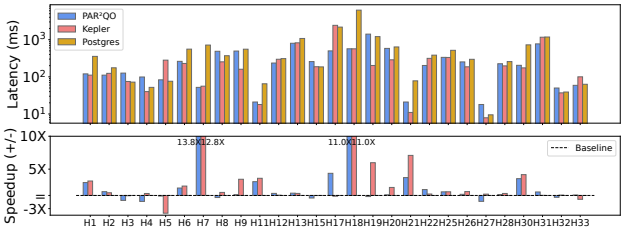
For DSB, we observe similar trends comparing PAR^{2}QO and Kepler. For CARVER, PAR^{2}QO achieves an average latency of 155ms, providing a 2.19× speedup over PostgreSQL, and 1.05× over Kepler. As for Gaussian, both PAR^{2}QO and Kepler achieve a 2.3× speedup over PostgreSQL. Neither PAR^{2}QO nor Kepler exhibit significant regression more than 1.2× across templates.



(a) CARVER Workload



(b) Uniform Workload



(c) Historical Workload

Figure 2: Query execution latency and speedup comparison on three workloads for JOB. The top panel of each figure compares the average latency (ms) on a log-scale, and the bottom shows the average speedup or regression over PostgreSQL (baseline).

5.2 Evaluation of PAR2QO Components

To gain deeper insights into the effectiveness of PAR^{2}QO, we evaluate the impact and contribution of each component in our system.

Impact of plan cache preparation. First, we examine how plan cache preparation affects PQO methods by evaluating the following

variants. Building on QueryLog′, we extend the approach by drawing $|\mathcal{L}|$ selectivity samples for each template and obtaining optimal plans using $\mathtt{Opt}(Q, \mathbf{s})$. Unlike PAR^{2}QO, these samples are derived from a space defined by $\widehat{\mathtt{Sel}}(Q)$ observed in the training workload, rather than from our clusters $\{\langle \mathcal{L}_i, \mathbf{s}_i, h_i \rangle\}$. Specifically, for each relevant subquery within the given template, we independently sample from $[\max(\hat{\mathbf{s}}_i), \min(\hat{\mathbf{s}}_i)]$. τ-approximate cover is retained for plan reduction, and at runtime, the plan with the lowest recost is selected. Results in Table 1 (labeled as **QueryLog′(sample)**) show that adding more samples from this *observed* selectivity space fails to mitigate QueryLog′'s performance regressions. In fact, performance degrades further on CARVER and Historical. Moreover, this variant exhibits fewer significant speedups and more severe regressions compared to QueryLog′.

We further evaluate a modified version of QueryLog′(sample) by replacing its plan set with the reduced robust plan set used in PAR^{2}QO, while the original runtime plan selection remains unchanged. Results, labeled as **BestCost** in Table 1, show significant improvements across all workloads. Notably, BestCost reverses the previous slowdowns: on CARVER, the −1.23× slowdown improves to a 1.56× speedup, surpassing Kepler's 1.07× but still trailing PAR^{2}QO's 1.96×. Similarly, on Uniform and Historical, while BestCost remains behind PAR^{2}QO and Kepler, it now achieves speedups over the PostgreSQL baseline. These findings, along with the performance of PAR^{2}QO and QueryLog′, highlight that while leveraging plans derived from queries and random selectivity samples in the training workload can approach near-optimizer performance, plans obtained through our selectivity probes are more performant. These comparisons underscore the importance of our constructed clusters and probe sampling mechanism for effective plan cache preparation.

Impact of runtime plan selection. Next, we evaluate how our runtime plan selection method contributes to performance. Since BestCost relies on the same robust plan cache as PAR^{2}QO but selects plans based on the lowest recost, the comparison between BestCost and PAR^{2}QO already demonstrates the superiority of our plan selection strategy. To further validate its effectiveness, we introduce additional baselines that leverage the same robust plan cache as PAR^{2}QO but employ different plan selection mechanisms.

- **BestCost (bound):** This variant extends BestCost by incorporating a safety check, similar to "cost check" in SRC[16]. It verifies whether the recost of a cached plan remains within a predefined λ-optimal threshold. If all cached plans exceed this criterion, the system falls back to invoking the optimizer. In experiments, we set $\lambda = 2$, as suggest in [16].
- **NearestSel**: Inspired by the selectivity-based plan reuse ("selectivity check") in SRC, this baseline simply selects plans based on the vector of selectivity estimates. At runtime, the nearest cached selectivity probe is searched using ℓ_2 distance, and the corresponding robust plan that covers this probe is selected.

Results are shown in Table 1, labeled as BestCost (bound) and NearestSel respectively. We observe that applying the cost bound check degrades BestCost's performance, indicating that the fallback mechanism does not guarantee a plan with truly optimal execution performance. This stems from the penalty caused by incorrect selectivity estimates $\widehat{\mathtt{Sel}}(Q)$: while the fallback selects the plan with the lowest estimated cost, it overlooks the potential that the selectivity estimation may be far from reality. As a result, the chosen plan may still perform poorly in practice due to estimation errors. In contrast, a plan from our reduced robust plan set may have a recost exceeding $\lambda \cdot \mathtt{Cost}^{\star}(Q)$, yet it can achieve better execution because of its robustness. Similarly, NearestSel fails to improve BestCost, due to the same issue—estimated selectivities can deviate significantly from the truth, rendering the distance check unreliable.

Impact of plan reduction. By default, PAR^{2}QO employs τ-Cover for plan reduction. To evaluate different reduction methods, we compare τ-Cover with **PAR^{2}QO (similarity)**, which reduces plan by cost profile similarity, as shown in Table 1. Results indicate that both methods achieve comparable performance, with PAR^{2}QO (similarity) introducing a slight slowdown, most notably a 1.06× degradation on Uniform. PAR^{2}QO (similarity) increases significant regression count on CARVER but reduces them on Uniform and Historical. We visualize the resulting plans for $C17$ under both reduction methods in [48]. Moreover, we notice that **PAR^{2}QO (w/o)** (no plan reduction) performs worse than PAR^{2}QO, confirming that plan reduction enhances robustness in selecting plans without sacrificing performance. We extend this ablation study to BestCost. The results, labeled as **BestCost (similarity)** and **BestCost (w/o)** in Table 1, show a more noticeable degradation over BestCost which employs τ-Cover for reduction, further demonstrate the inherent robustness provided by this natural set cover method. On JOB, PAR^{2}QO generates 88 candidate plans per template. τ-Cover reduces $|\mathcal{P}|$ to 8. Additionally, we observe that, on average, a set of 9 robust plans is sufficient to "τ-cover" all probes. Based on these findings, we recommend $k \leq 0.2 \cdot |\mathcal{P}|$, with a minimum of 10 plans to maintain diversity. The adaptive selection of k and the theoretical analysis of $|\mathcal{P}|$ required for τ-cover are interesting directions for future work.

Model preparation time. Besides query execution performance, another significant advantage of PAR^{2}QO is its substantially lower end-to-end[2] model preparation time. PAR^{2}QO operates on an error model profiled from the training workload (Section 3), which incurs a one-time upfront cost across all templates. For all three workloads on JOB, this totals 20 hours (less than 13 minutes on average per template). As discussed in Section 3, preparing $\mathfrak{P}$ requires $|\mathcal{L}|$ calls to $\mathtt{Opt}$ and $|\mathcal{P}| \cdot |\mathcal{L}|$ calls to $\mathtt{Cost}$. On average, n=50 results in $|\mathcal{L}|$=300 and $|\mathcal{P}|$=88. The entire penalty-profiling time is 8.5 minutes on average. The subsequent reduction is highly efficient, completing in under 500ms per template. Thus, the E2E model preparation time for PAR^{2}QO on JOB averages 21.5 minutes per template. In contrast, Kepler requires real query executions, leading to an average training time over 6.5 hours per template—more than 18× slower than PAR^{2}QO. Similarly, as discussed in Section 2.2, the expensive sensitivity analysis required for each $\langle \pi_i, \mathbf{s}_i \rangle \in \mathcal{C}$ results in an average of 2.4 hours (including $\mathcal{E}$ construction) for PARQO′, which is 6.7× slower than PAR^{2}QO. On DSB, Kepler takes 4 hours per template, while PAR^{2}QO requires only 9.5 minutes on average.

Impact of training size. For those templates where Kepler exhibits sever regressions, e.g. $C3$-4, $U4$, $H5$, and $C10$, as shown in

[2] "End-to-end" time includes the period from receiving the training workload to completing model setup and being ready to generate plans for new queries.

Figure 2, we expand its original training set from 50 to 400 unique queries. This increases Kepler's preparation time substantially—up to 120 hours per template—while the performance impact varies across templates. Some regressions improve via additional training: $C4$ improves from $-3.71\times$ to $-1.50\times$, $U4$ from $-4.0\times$ to a $1.31\times$ speedup, and $H5$ from $-3.70\times$ to $-1.30\times$. However, $C10$ and $C3$ remain unchanged at their original regressions of $-5.04\times$ and $-3.46\times$. These results suggest that while a larger training workload can mitigate some regressions, Kepler's effectiveness heavily depends on template and workload characteristics, offering no consistent improvement. Notably, while Kepler outperforms PAR^{2}QO on $U4$ after expansion, PAR^{2}QO, using only the original 50 training queries, is still better on all other templates. We also expanded the training sets for PAR^{2}QO, but the performance (still using 50 as the probe size per cluster) showed minimal change, further demonstrating its effectiveness even with a small training size.

Impact of probe size. To evaluate the effect of a larger probe size, we increase it to 200 for the three templates with the most significant regressions in Figure 2: $H4$, $H3$, and $H27$. After expending, $H4$ improves from $-1.90\times$ to $1.41\times$ speedup, and $H3$ from $-1.75\times$ to $-1.46\times$. However, $H27$ remains unchanged at $-1.58\times$, showing that a larger probe size does not always resolve performance regressions. More probes also increase model preparation time, as each plan's profiling becomes more computationally expensive and additional candidate plans may be obtained. Furthermore, inference time may also increase (discussed later). To balance performance, preparation cost, and inference efficiency, we recommend using 20–100 samples.

Model size and inference time. Beyond query execution performance, practical PQO methods must account for physical model size, inference efficiency, and practical deployment constraints. On average across JOB templates, PAR^{2}QO consists of a 0.2MB error model $\mathcal{E}$ and a cache—including $\mathfrak{P}$, $\mathcal{L}$, and $\mathcal{P}$—of 1.8MB (2.6MB before plan reduction), totaling approximately 2.1MB. For inference speed, PAR^{2}QO selects plan in an average of 26ms for one query (31ms before plan reduction), outperforming PostgreSQL's average planning time of 67ms. In comparison, Kepler, which uses simple feed-forward neural networks, maintains a more compact model of under 1MB and achieves significantly faster inference, taking less than 1ms per query. Both PAR^{2}QO (as discussed in Section 2.1) and Kepler can be easily integrated into existing DBMS.

5.3 Plan Robustness Evaluation on Data Shifts

The training and testing in previous experiments were conducted on the same static snapshot of $\mathcal{D}$, where underlining data distributions remained unchanged. However, true robustness requires plans to maintain good performance as data distributions evolve, preventing significant degradation in execution. In this experiment, we simulate real-world scenarios with shifting data distributions, which can introduce unforeseen variations in selectivity uncertainties.

We create multiple database instances derived from the original IMDB dataset and use three slicing methods to capture varying degrees of data distribution shifts. First, ***category-slicing***[47] splits data by item categories (*kind_type.kind*), representing dramatic distribution shifts[3]. Second, ***time-slicing***[47] applies a 20% sliding window on *production_year*, generating 9 instances. Adjacent instances share overlapping data, while distant ones exhibit larger differences, capturing smooth transitions and gradual shifts. Third, ***random-slicing*** randomly samples 20% of *title* along with associated data from other tables, maintaining the overall distribution with minimal selectivity shifts, representing the mildest scenario.

[3]Table sizes can vary up to 51× (e.g. *title* in "Movie" vs. "Video game"), and execution times may fluctuate up to 176× (e.g. C17 on "Movie" vs. "Video game").

Table 2: Comparison of workload effectiveness in training PQO model across different methods on JOB.

		CARVER	Uniform	Historical
PostgreSQL	Avg latency	552 ms	552 ms	552 ms
PAR^{2}QO	Avg latency	315 (1.75×)	407 (1.35×)	400 (1.38×)
	↑ > 1.2×	15/33	12/33	15/33
	↓ > 1.2×	2/33	8/33	7/33
	↓ > 2×	1/33	1/33	3/33
Kepler	Avg latency	555 (-1.01×)	553 (-1.00×)	909 (-1.65×)
	↑ > 1.2×	15/33	8/33	9/33
	↓ > 1.2×	8/33	14/33	10/33
	↓ > 2×	3/33	5/33	5/33

For each dataset-slicing method, we synthesize three workloads (CARVER, Uniform, and Historical) by the same procedures in Section 5.1. PAR^{2}QO (and Kepler) selects plans using information from one base instance alone, and these plans are evaluated on remaining instances. Note that PAR^{2}QO's error model $\mathcal{E}$ is also derived by the base $\mathcal{D}$ only. PostgreSQL's plan, selected by refreshed statistics after the data shifting, serves as a baseline. Due to the large number of templates, we select four representative ones: 4, 14, 17, and 19, based on various query complexities and their original performance in Table 1, with detailed selection criteria provided in [48]. Due to page limits, we also put detailed results in [48]. In summary, across different slicing methods, PAR^{2}QO consistently delivers stable performance with minimal severe regressions, whereas Kepler experiences multiple cases of such degradation. These results underscore PAR^{2}QO's robustness across diverse data shifts.

5.4 Workload Effectiveness Evaluation

To evaluate the workload effectiveness, we train each PQO method exclusively on one of three workloads—CARVER, Uniform, or Historical—and then test using a *mixed* workload sampled from all three. This mixed testing workload simulates a diverse and unpredictable distribution of binding parameters and selectivity variations, including cases not explicitly represented in the training workload—a scenario commonly encountered in the real world. The goal is to determine which training workload best optimizes model performance under these conditions, in the meanwhile, assessing how well a model adapts to unforeseen workload distributions.

The results are shown in Table 2. For PAR^{2}QO, training with CARVER yields better performance than using Uniform or Historical workloads. Specifically, PAR^{2}QO trained on CARVER achieves a 1.75× speedup over PostgreSQL's baseline, whereas training on Uniform and Historical results in lower speedups of 1.35× and 1.38×, respectively. Moreover, CARVER leads to the fewest templates experiencing significant or severe performance regressions. As a comparison, for Kepler, CARVER and Uniform offer comparable average query latency, both outperforming Historical. However, CARVER results in fewer significant or severe regressions than Uniform and Historical, demonstrating its effectiveness in enhancing the generality and stability of PQO models. To help ensure that

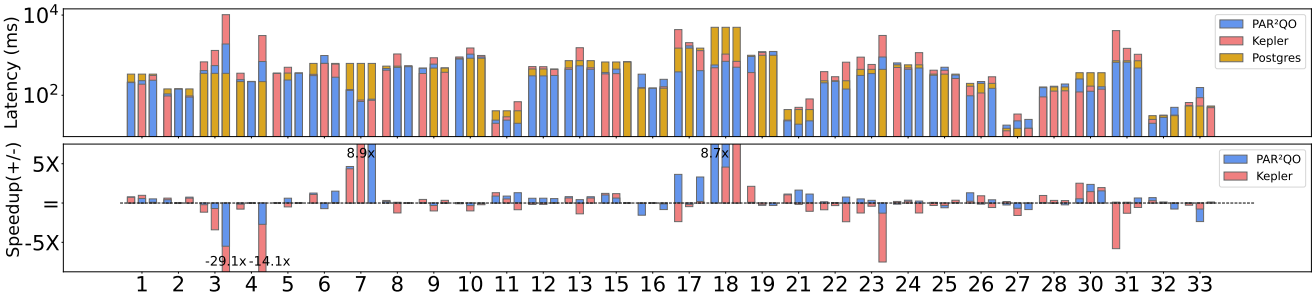


Figure 3: Workload effectiveness evaluation on JOB. Each query template shows three bars (left to right: CARVER, Uniform, Historical), representing different workloads used for training. Each bar shows three overlapping segments, corresponding to PAR^{2}QO, Kepler and PostgreSQL. The top panel compares execution latency; the lower shows the speedup/regression of PAR^{2}QO and Kepler relative to PostgreSQL.

our conclusion is not biased by outliers, we reexamine the results by excluding the single template causing the largest performance decline when trained on each workload, see [48] for details.

From Table 2, the performance of PAR^{2}QO is further validated. PAR^{2}QO consistently outperforms Kepler and PostgreSQL across all workloads, regardless of what training workload is used. In contrast, Kepler exhibits a significant degradation of $-1.65\times$ when trained on Historical. Interestingly, although Kepler performs well on Historical in Table 1, it lacks generalization and struggles to consistently provide efficient plans for unforeseen queries. We also provide detailed template level performance in Figure 3[4]. Similar to Figure 2, Kepler achieves strong performance on some templates (e.g., $C19$ and $C30$), but also suffers severe degradations in others (e.g., $H3$-4, $H23$, and $C31$). In contrast, PAR^{2}QO effectively avoids such extreme regressions, highlighting that PAR^{2}QO's generality and stability advantage in maintaining plan efficiency and preventing extreme degradation under unforeseen workload distributions.

On DSB, a similar pattern emerges when testing with a mixture of CARVER and Gaussian. PAR^{2}QO achieves a $2.28\times$ speedup over PostgreSQL when trained on CARVER, surpassing it when trained on Gaussian ($1.67\times$). In comparison, Kepler achieves a $2.25\times$ speedup when trained on CARVER compared to $1.68\times$ on Gaussian.

6 RELATED WORK

Parametric Query Optimization. PQO has been extensively studied over the years[23, 24]. The primary focus of traditional PQO methods is minimizing the optimizer's invocation through plan caching and reuse strategies, ensuring the plan's cost remains acceptable while avoiding unnecessary re-optimization[2, 6, 16]. Different approaches have been explored to address this challenge, including caching only *one* plan with the lowest expected cost for the entire selectivity space[1, 6, 7], caching a *set* of plans [16, 19, 25], or progressively caching multiple plans over time[2, 5, 28]. Some works propose to build density maps through query clustering to better capture selectivity variations[2, 19], or building a "plan diagram" as a color-code visualization[12, 38] to provide a structured view of the selectivity space and how different plans dominate various regions. For a more comprehensive review of traditional PQO techniques, we refer readers to [11]. Recent work, SCR[16] applies an online selectivity and cost check to evaluate if a cached plan can be reused guaranteeing the bound on cost sub-optimality, and if a new plan needs to be added into the cache. Most recent efforts, including QueryLog[41] and Kepler[14], extend PQO by incorporating execution performance optimization and learning from query executions. QueryLog formalizes PQO problem as a two-step decision process, generating candidate plans and selecting the most suitable one. Kepler and PAR^{2}QO adhere to this structure. Since Kepler and Querylog have been thoroughly discussed and compared with PAR^{2}QO in Section 3.4, we omit further details here.

Robust query optimization. Traditional query optimizers often select plans that perform poorly in practice, with cardinality estimation errors being a primary cause of such degradations [30, 31]. RQO aims to select robust plans that remain stable despite inaccuracies in cardinality estimates[34, 49]. To quantify these uncertainties and guide robust plan selection, prior research assumes cardinality as a probability density function [3, 8], a uniform distribution over the entire space [1], or within a bounding box [4, 18]. More recent work has developed advanced plan selection techniques: [15, 37, 42] considers worst-case sub-optimality, [45] defines robustness metrics based on cost slope and integral, [13, 46] computes optimality or near-optimality ranges as robustness indicators, [33] selects plans based on learned execution time distribution, and [47] prioritizes plans using expected penalty. Another line of research explores *adaptive processing*, where runtime observations guide dynamic plan switching to avoid poor plan executions [15, 17, 40, 43, 45]. For a comprehensive survey on robust query processing, see [21]. Previous methods [9, 12–14, 38, 39, 41, 46] implicitly consider robustness when pruning candidate plans, while PAR^{2}QO takes a step further by directly integrating plan robustness into PQO.

7 CONCLUSION AND FUTURE WORK

Selecting a PQO method in practice involves a trade-off. Kepler can sometimes find the fastest plans with low inference latency but requires expensive model training and is prone to severe regressions. QueryLog is easy to set up but may fail to improve execution performance beyond the optimizer. By integrating penalty-aware robustness into PQO, PAR^{2}QO achieves a better balance between efficiency, stability, and practicality.

Future work includes reusing computations across templates and adapting the error model and cache dynamically for evolving data and database changes. Additionally, improving debugging tools for workload performance and refining robust plan selection beyond optimal plans remain open challenges.

[4]Templates 14, 20, and 29 are excluded in Figure 3, as both PAR^{2}QO and Kepler's results show no significant speedup/regression beyond $1.2\times$.

REFERENCES

[1] M. Abhirama, Sourjya Bhaumik, Atreyee Dey, Harsh Shrimal, and Jayant R. Haritsa. 2010. On the Stability of Plan Costs and the Costs of Plan Stability. *Proc. VLDB Endow.* 3, 1 (2010), 1137–1148. https://doi.org/10.14778/1920841.1920983

[2] Günes Aluç, David DeHaan, and Ivan T. Bowman. 2012. Parametric Plan Caching Using Density-Based Clustering. In *IEEE 28th International Conference on Data Engineering (ICDE 2012), Washington, DC, USA (Arlington, Virginia), 1-5 April, 2012*, Anastasios Kementsietsidis and Marcos Antonio Vaz Salles (Eds.). IEEE Computer Society, 402–413. https://doi.org/10.1109/ICDE.2012.57

[3] Brian Babcock and Surajit Chaudhuri. 2005. Towards a Robust Query Optimizer: A Principled and Practical Approach. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Baltimore, Maryland, USA, June 14-16, 2005*, Fatma Özcan (Ed.). ACM, 119–130. https://doi.org/10.1145/1066157.1066172

[4] Shivnath Babu, Pedro Bizarro, and David J. DeWitt. 2005. Proactive Re-optimization. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Baltimore, Maryland, USA, June 14-16, 2005*, Fatma Özcan (Ed.). ACM, 107–118. https://doi.org/10.1145/1066157.1066171

[5] Pedro Bizarro, Nicolas Bruno, and David J. DeWitt. 2009. Progressive Parametric Query Optimization. *IEEE Trans. Knowl. Data Eng.* 21, 4 (2009), 582–594. https://doi.org/10.1109/TKDE.2008.160

[6] Surajit Chaudhuri, Hongrae Lee, and Vivek R. Narasayya. 2010. Variance aware optimization of parameterized queries. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2010, Indianapolis, Indiana, USA, June 6-10, 2010*, Ahmed K. Elmagarmid and Divyakant Agrawal (Eds.). ACM, 531–542. https://doi.org/10.1145/1807167.1807226

[7] Francis C. Chu, Joseph Y. Halpern, and Johannes Gehrke. 2002. Least Expected Cost Query Optimization: What Can We Expect?. In *Proceedings of the Twenty-first ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, June 3-5, Madison, Wisconsin, USA*, Lucian Popa, Serge Abiteboul, and Phokion G. Kolaitis (Eds.). ACM, 293–302. https://doi.org/10.1145/543613.543651

[8] Francis C. Chu, Joseph Y. Halpern, and Praveen Seshadri. 1999. Least Expected Cost Query Optimization: An Exercise in Utility. In *Proceedings of the Eighteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, May 31 - June 2, 1999, Philadelphia, Pennsylvania, USA*, Victor Vianu and Christos H. Papadimitriou (Eds.). ACM Press, 138–147. https://doi.org/10.1145/303976.303990

[9] Atreyee Dey, Sourjya Bhaumik, Harish Doraiswamy, and Jayant R. Haritsa. 2008. Efficiently approximating query optimizer plan diagrams. *Proc. VLDB Endow.* 1, 2 (2008), 1325–1336. https://doi.org/10.14778/1454159.1454173

[10] Bailu Ding, Surajit Chaudhuri, Johannes Gehrke, and Vivek R. Narasayya. 2021. DSB: A Decision Support Benchmark for Workload-Driven and Traditional Database Systems. *Proc. VLDB Endow.* 14, 13 (2021), 3376–3388. https://doi.org/10.14778/3484224.3484234

[11] Bailu Ding, Vivek Narasayya, and Surajit Chaudhuri. 2024. Extensible Query Optimizers in Practice. *Foundations and Trends® in Databases* 14, 3-4 (2024), 186–402. https://doi.org/10.1561/1900000077

[12] Harish Doraiswamy, Pooja N. Darera, and Jayant R. Haritsa. 2007. On the Production of Anorexic Plan Diagrams. In *Proceedings of the 33rd International Conference on Very Large Data Bases, University of Vienna, Austria, September 23-27, 2007*, Christoph Koch, Johannes Gehrke, Minos N. Garofalakis, Divesh Srivastava, Karl Aberer, Anand Deshpande, Daniela Florescu, Chee Yong Chan, Venkatesh Ganti, Carl-Christian Kanne, Wolfgang Klas, and Erich J. Neuhold (Eds.). ACM, 1081–1092. http://www.vldb.org/conf/2007/papers/research/p1081-d.pdf

[13] Harish Doraiswamy, Pooja N. Darera, and Jayant R. Haritsa. 2008. Identifying robust plans through plan diagram reduction. *Proc. VLDB Endow.* 1, 1 (2008), 1124–1140. https://doi.org/10.14778/1453856.1453976

[14] Lyric Doshi, Vincent Zhuang, Gaurav Jain, Ryan Marcus, Haoyu Huang, Deniz Altinbüken, Eugene Brevdo, and Campbell Fraser. 2023. Kepler: Robust Learning for Parametric Query Optimization. *Proc. ACM Manag. Data* 1, 1 (2023), 109:1–109:25. https://doi.org/10.1145/3588963

[15] Anshuman Dutt and Jayant R Haritsa. 2014. Plan bouquets: query processing without selectivity estimation. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 1039–1050.

[16] Anshuman Dutt, Vivek R. Narasayya, and Surajit Chaudhuri. 2017. Leveraging Re-costing for Online Optimization of Parameterized Queries with Guarantees. In *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017, Chicago, IL, USA, May 14-19, 2017*, Semih Salihoglu, Wenchao Zhou, Rada Chirkova, Jun Yang, and Dan Suciu (Eds.). ACM, 1539–1554. https://doi.org/10.1145/3035918.3064040

[17] Anshuman Dutt, Sumit Neelam, and Jayant R Haritsa. 2014. QUEST: An exploratory approach to robust query processing. *Proceedings of the VLDB Endowment* 7, 13 (2014), 1585–1588.

[18] Belgin Ergenc, Franck Morvan, and Abdelkader Hameurlain. 2007. Robust Placement of Mobile Relational Operators for Large Scale Distributed Query Optimization. In *Eighth International Conference on Parallel and Distributed Computing, Applications and Technologies (PDCAT 2007), 3-6 December 2007, Adelaide, Australia*, David S. Munro, Hong Shen, Quan Z. Sheng, Henry Detmold, Katrina Falkner, Cruz Izu, Paul D. Coddington, Bradley Alexander, and Si-Qing Zheng (Eds.). IEEE Computer Society, 227–235. https://doi.org/10.1109/PDCAT.2007.53

[19] Antara Ghosh, Jignashu Parikh, Vibhuti S. Sengar, and Jayant R. Haritsa. 2002. Plan Selection Based on Query Clustering. In *Proceedings of 28th International Conference on Very Large Data Bases, VLDB 2002, Hong Kong, August 20-23, 2002*. Morgan Kaufmann, 179–190. https://doi.org/10.1016/B978-155860869-6/50024-X

[20] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *Proc. VLDB Endow.* 15, 4 (2021), 752–765. https://doi.org/10.14778/3503585.3503586

[21] Jayant R. Haritsa. 2024. Robust Query Processing: A Survey. *Found. Trends Databases* 15, 1 (2024), 1–114. https://doi.org/10.1561/1900000089

[22] Dawei Huang, Dong Young Yoon, Seth Pettie, and Barzan Mozafari. 2019. Join on Samples: A Theoretical Guide for Practitioners. *Proc. VLDB Endow.* 13, 4 (2019), 547–560. https://doi.org/10.14778/3372716.3372726

[23] Arvind Hulgeri and S. Sudarshan. 2002. Parametric Query Optimization for Linear and Piecewise Linear Cost Functions. In *Proceedings of 28th International Conference on Very Large Data Bases, VLDB 2002, Hong Kong, August 20-23, 2002*. Morgan Kaufmann, 167–178. https://doi.org/10.1016/B978-155860869-6/50023-8

[24] Yannis E. Ioannidis, Raymond T. Ng, Kyuseok Shim, and Timos K. Sellis. 1992. Parametric Query Optimization. In *18th International Conference on Very Large Data Bases, August 23-27, 1992, Vancouver, Canada, Proceedings*, Li-Yan Yuan (Ed.). Morgan Kaufmann, 103–114. http://www.vldb.org/conf/1992/P103.PDF

[25] Yannis E. Ioannidis, Raymond T. Ng, Kyuseok Shim, and Timos K. Sellis. 1997. Parametric Query Optimization. *VLDB J.* 6, 2 (1997), 132–151. https://doi.org/10.1007/S007780050037

[26] Srikanth Kandula, Anil Shanbhag, Aleksandar Vitorovic, Matthaios Olma, Robert Grandl, Surajit Chaudhuri, and Bolin Ding. 2016. Quickr: Lazily Approximating Complex AdHoc Queries in BigData Clusters. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, Fatma Özcan, Georgia Koutrika, and Sam Madden (Eds.). ACM, 631–646. https://doi.org/10.1145/2882903.2882940

[27] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter A. Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *9th Biennial Conference on Innovative Data Systems Research, CIDR 2019, Asilomar, CA, USA, January 13-16, 2019, Online Proceedings*. www.cidrdb.org. http://cidrdb.org/cidr2019/papers/p101-kipf-cidr19.pdf

[28] Allison W. Lee and Mohamed Zaït. 2008. Closing the query processing loop in Oracle 11g. *Proc. VLDB Endow.* 1, 2 (2008), 1368–1378. https://doi.org/10.14778/1454159.1454178

[29] Kukjin Lee, Anshuman Dutt, Vivek R. Narasayya, and Surajit Chaudhuri. 2023. Analyzing the Impact of Cardinality Estimation on Execution Plans in Microsoft SQL Server. *Proc. VLDB Endow.* 16, 11 (2023), 2871–2883. https://doi.org/10.14778/3611479.3611494

[30] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proc. VLDB Endow.* 9, 3 (2015), 204–215. https://doi.org/10.14778/2850583.2850594

[31] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proc. VLDB Endow.* 9, 3 (2015), 204–215. https://doi.org/10.14778/2850583.2850594

[32] Viktor Leis, Bernhard Radke, Andrey Gubichev, Atanas Mirchev, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2018. Query optimization through the looking glass, and what we found running the Join Order Benchmark. *VLDB J.* 27, 5 (2018), 643–668. https://doi.org/10.1007/S00778-017-0480-7

[33] Yifan Li, Xiaohui Yu, Nick Koudas, Shu Lin, Calvin Sun, and Chong Chen. 2023. dbET: Execution Time Distribution-based Plan Selection. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.

[34] Guy M. Lohman. 2017. Query Optimization - Are We There Yet?. In *Datenbanksysteme für Business, Technologie und Web (BTW 2017), 17. Fachtagung des GI-Fachbereichs „Datenbanken und Informationssysteme" (DBIS), 6.-10. März 2017, Stuttgart, Germany, Proceedings (LNI)*, Bernhard Mitschang, Daniela Nicklas, Frank Leymann, Harald Schöning, Melanie Herschel, Jens Teubner, Theo Härder, Oliver Kopp, and Matthias Wieland (Eds.), Vol. P-265. GI, 25–26. https://dl.gi.de/handle/20.500.12116/646

[35] Satoshi Nagayasu. 2023. pg_hint_plan. https://github.com/ossc-db/pg_hint_plan.

[36] Raghunath Othayoth Nambiar and Meikel Poess. 2006. The Making of TPC-DS. In *Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12-15, 2006*, Umeshwar Dayal, Kyu-Young Whang, David B. Lomet, Gustavo Alonso, Guy M. Lohman, Martin L. Kersten, Sang Kyun Cha, and Young-Kuk Kim (Eds.). ACM, 1049–1058. http://dl.acm.org/citation.cfm?id=1164217

[37] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2012. Worst-case optimal join algorithms: [extended abstract]. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, PODS 2012, Scottsdale, AZ, USA, May 20-24, 2012*, Michael Benedikt, Markus Krötzsch,

and Maurizio Lenzerini (Eds.). ACM, 37–48. https://doi.org/10.1145/2213556.2213565

[38] Naveen Reddy and Jayant R. Haritsa. 2005. Analyzing Plan Diagrams of Database Query Optimizers. In *Proceedings of the 31st International Conference on Very Large Data Bases, Trondheim, Norway, August 30 - September 2, 2005*, Klemens Böhm, Christian S. Jensen, Laura M. Haas, Martin L. Kersten, Per-Åke Larson, and Beng Chin Ooi (Eds.). ACM, 1228–1240. http://www.vldb.org/archives/website/2005/program/paper/fri/p1228-reddy.pdf

[39] Naveen Reddy and Jayant R. Haritsa. 2005. Analyzing Plan Diagrams of Database Query Optimizers. In *Proceedings of the 31st International Conference on Very Large Data Bases, Trondheim, Norway, August 30 - September 2, 2005*, Klemens Böhm, Christian S. Jensen, Laura M. Haas, Martin L. Kersten, Per-Åke Larson, and Beng Chin Ooi (Eds.). ACM, 1228–1240. http://www.vldb.org/archives/website/2005/program/paper/fri/p1228-reddy.pdf

[40] Immanuel Trummer, Junxiong Wang, Ziyun Wei, Deepak Maram, Samuel Moseley, Saehan Jo, Joseph Antonakakis, and Ankush Rayabhari. 2021. Skinnerdb: Regret-bounded query evaluation via reinforcement learning. *ACM Transactions on Database Systems (TODS)* 46, 3 (2021), 1–45.

[41] Kapil Vaidya, Anshuman Dutt, Vivek R. Narasayya, and Surajit Chaudhuri. 2021. Leveraging Query Logs and Machine Learning for Parametric Query Optimization. *Proc. VLDB Endow.* 15, 3 (2021), 401–413. https://doi.org/10.14778/3494124.3494126

[42] Srinivas Karthik Venkatesh, Jayant R. Haritsa, Sreyash Kenkre, and Vinayaka Pandit. 2018. A Concave Path to Low-overhead Robust Query Processing. *Proc. VLDB Endow.* 11, 13 (2018), 2183–2195. https://doi.org/10.14778/3275366.3275368

[43] Ziyun Wei and Immanuel Trummer. 2024. ROME: Robust Query Optimization via Parallel Multi-Plan Execution. *Proc. ACM Manag. Data* 2, 3 (2024), 170. https://doi.org/10.1145/3654973

[44] David P Williamson and David B Shmoys. 2011. *The design of approximation algorithms.* Cambridge university press.

[45] Florian Wolf, Michael Brendle, Norman May, Paul R. Willems, Kai-Uwe Sattler, and Michael Grossniklaus. 2018. Robustness Metrics for Relational Query Execution Plans. *Proc. VLDB Endow.* 11, 11 (2018), 1360–1372. https://doi.org/10.14778/3236187.3236191

[46] Florian Wolf, Norman May, Paul R. Willems, and Kai-Uwe Sattler. 2018. On the Calculation of Optimality Ranges for Relational Query Execution Plans. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 663–675. https://doi.org/10.1145/3183713.3183742

[47] Haibo Xiu, Pankaj K. Agarwal, and Jun Yang. 2024. PARQO: Penalty-Aware Robust Plan Selection in Query Optimization. *Proc. VLDB Endow.* 17, 13 (2024), 4627–4640. https://doi.org/10.14778/3704965.3704971

[48] Haibo Xiu, Yang Li, Qianyu Yang, Pankaj K. Agarwal, and Jun Yang. 2025. (Full Version Paper). https://github.com/Hap-Hugh/PAR2QO/blob/main/par2qo.pdf

[49] Shaoyi Yin, Abdelkader Hameurlain, and Franck Morvan. 2015. Robust query optimization methods with respect to estimation errors: A survey. *ACM Sigmod Record* 44, 3 (2015), 25–36.

A DETAILED ELABORATION OF EXAMPLE 1 IN SECTION 4

Here, we provide a detailed procedure elaborating on Example 1 in Section 4.1. Consider template 17 below from the JOB benchmark [31].

```
SELECT MIN(n.name) AS member_in_charnamed_american_movie,
       MIN(n.name) AS a1
FROM cast_info AS ci, company_name AS cn, keyword AS k,
     movie_companies AS mc, movie_keyword AS mk,
     name AS n, title AS t
WHERE cn.country_code ='XXX' AND k.keyword ='XXX' AND n.name LIKE
       'XXX'
  AND n.id = ci.person_id AND ci.movie_id = t.id
  AND t.id = mk.movie_id AND mk.keyword_id = k.id
  AND t.id = mc.movie_id AND mc.company_id = cn.id
  AND ci.movie_id = mc.movie_id AND ci.movie_id = mk.movie_id
  AND mc.movie_id = mk.movie_id;
```

As described, the two-component mixture distribution considers both small subquery fragments (*querylets*) and the overall query. The first component, P_1, consists of three two-table querylets, which are $mc \bowtie cn^{\sigma}$, $k^{\sigma} \bowtie mk$, and $n^{\sigma} \bowtie ci$ (the superscript σ denotes the presence of at least one local selection condition on a table). P_1 is then defined as the product distribution of the parameter distribution of each querylet. The second component P_2, represents the full query, consisting of only one template which joins all tables in template 17 with selection conditions on *cn.country_code*, *k.keyword*, and *n.name* respectively.

B JUSTIFICATION FOR BIAS CORRECTION

Given $\widehat{\mathsf{Sel}}(Q)$, we aim to estimated the expected penalty $\mathsf{E}[\mathsf{Penalty}(\pi, \mathbf{S}) \mid \widehat{\mathsf{Sel}}(Q)]$ for each π using $\mathfrak{P}$ and probe locations $\mathcal{L}$. This expectation is derived over the target distribution conditioned on the new estimates $f(\mathbf{s}|\widehat{\mathsf{Sel}}(Q))$, which is obtained from the error model $\mathcal{E}$.

Suppose we have an *overall* probability density distribution of selectivity $\mathbf{F}(\mathbf{y})$, and R samples $\{\mathbf{y}_1, \mathbf{y}_2, ..., \mathbf{y}_R\}$ drawn from $\mathbf{F}(\mathbf{y})$. Using importance sampling, we estimate the target expectation as:

$$\mathsf{E}[\mathsf{Penalty}(\pi, \mathbf{S}) \mid \widehat{\mathsf{Sel}}(Q)] = \sum_{i=1}^{R} \frac{f(\mathbf{y}|\widehat{\mathsf{Sel}}(Q))}{\mathbf{F}(\mathbf{y})} \cdot \mathfrak{P}[\pi, \mathbf{y}].$$

These R samples can be divided into m independent clusters $\{M_1, M_2, .., M_m\}$, where the i-th cluster M_i contains $R_i = h_i \cdot N$ samples, with $h_i \geq 1$ representing the density of cluster M_i and N is a constant. Thus, $R = \sum_{i=1}^{m} h_i \cdot N$.

While $\mathbf{F}(\mathbf{y})$ is unknown, we know the probability of $\mathbf{y}$ being sampled within the i-th cluster, $\mathbf{Q}_i(\mathbf{y})$. Since clusters are independent, the probability of selecting the i-th cluster is $\frac{h_i}{\sum_{i=1}^{m} h_i}$. Since a sample $\mathbf{y}$ can be drawn from *any* cluster in m clusters, the probability of $\mathbf{y}$ being sampled from the $\mathbf{F}(\mathbf{y})$ can be calculated as the cumulated probability of being sampled from each cluster: $\sum_{i=1}^{m} \frac{h_i}{\sum_{i=1}^{m} h_i} \cdot \mathbf{Q}_i(\mathbf{y}) = \frac{1}{R} \sum_{i=1}^{m} h_i \mathbf{Q}_i(\mathbf{y})$. And finally, after dropping terms that do not affect plan ranking, the expected penalty is estimated as:

$$\sum_{i=1}^{R} \frac{f(\mathbf{y}|\widehat{\mathsf{Sel}}(Q))}{\sum_{i=1}^{m} h_i \mathbf{Q}_i(\mathbf{y})} \cdot \mathfrak{P}[\pi, \mathbf{y}].$$

If $R = mn$, this becomes,

$$\mathsf{E}[\mathsf{Penalty}(\pi, \mathbf{S}) \mid \widehat{\mathsf{Sel}}(Q)] = \sum_{i=1}^{m} \sum_{j=1}^{n} \frac{f(\mathbf{y}|\widehat{\mathsf{Sel}}(Q))}{h_i \mathbf{Q}_i(\mathbf{y})} \cdot \mathfrak{P}[\pi, \mathbf{y}].$$

This aligns with our bias correction in Section 3.3 by simply replacing $\mathbf{y}$ with $s_{i,j}$ and $\mathbf{Q}_i(\mathbf{y})$ with $f(\mathbf{s}_{i,j}|\mathbf{s}_i)$.

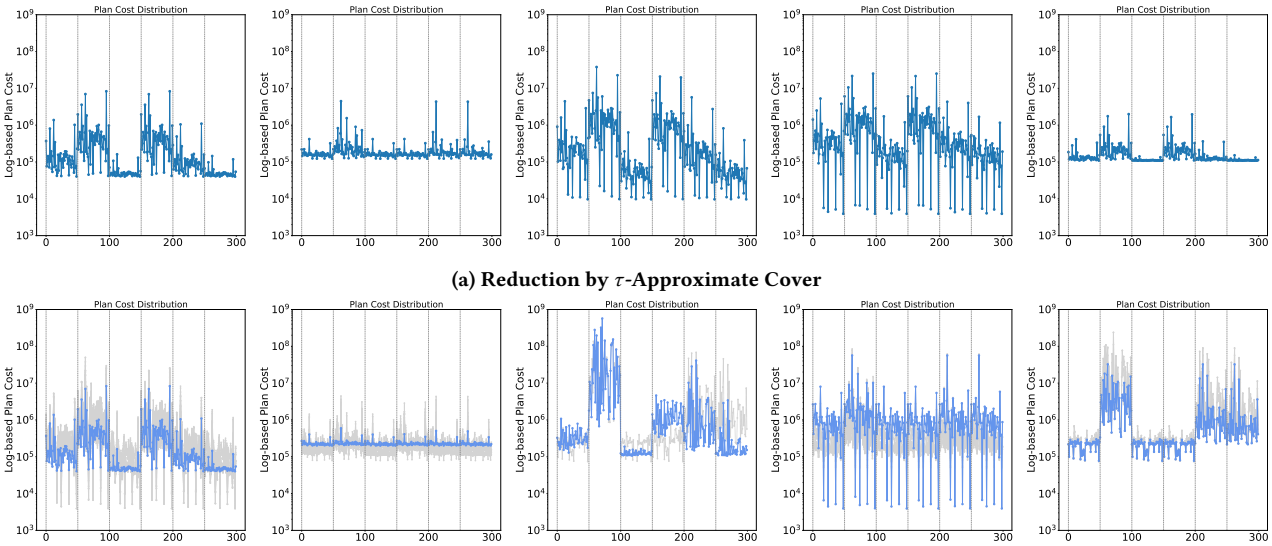
C PLAN VISUALIZATION AFTER PLAN REDUCTION

Figure 4 visualizes the cost distribution of plans after applying different reduction methods across selectivity probes for JOB template 17, with CARVER as the underlined workload. For the λ-Cover reduction method, we present the top five queries with the largest coverage range. For similarity-based reduction, we set the target plan size $k = 5$. The remaining plans from both methods show significant overlaps. Using the set by either reduction method as the plan cache achieves over a 4× speedup compared to PostgreSQL, demonstrating their effectiveness in selecting robust plans. Additionally, for plans reduced by cost profile similarity, we observe that those plans within the same group exhibit highly similar cost distributions, while plans from different groups display distinct behavior. These observations suggest the JS divergence is a suitable metric for measuring plan similarity, and the k-center greedy method can effectively preserve plan diversity.

D DETAILED CONFIGURATIONS IN EXPERIMENTS

Kepler setup. In our experiments, we use three generations of RCE (Row Count Evolution)[14] to perturb all sub-queries within the given template. For the exponent base and range, we also refer to the default value: $b = 10$ and $m = 2$. On average, Kepler populates 199 candidate plans for JOB and 130 for DSB. To prune these candidate plans into a small plan set (referred to as "plan covers" in [14]) that ensures near-optimal execution latency for all training queries, we execute each candidate plan three times per training query. Then collect the *fast* plans using the default greedy "Set Cover" method. Since the initial training workload is small, we set δ=0.01, ensuring that all queries will be covered by the pruned plans. When training with 400 queries, as in Section 5.2, the resulting set of plans will cover 396 queries ($400 \cdot (1 - \delta)$). We also follow the timeout policy to avoid executing poor plans. After this pruning, an average of 3.4 plans per template remain for JOB, and 1.23 for DSB. Given the small training size, we observe that Kepler's confidence threshold is consistently below 0.6. Setting the threshold to ≥ 0.6 forces all predictions to fall back to the optimizer. Therefore, following the recommendations in [14], we set the confidence threshold to 0.4.

PARQO′ setup. PARQO′ and PAR^{2}QO use the same error model. For each respective selectivities $\mathbf{s}_i$ (referred to as "anchors" in [47]), we use Sobols method with 128 seeds to identify sensitive dimensions and by default the top 3 sensitive dimensions are selected to find robust plans. A same θ_{KL} threshold (ln 200 as recommended in [47]) is used when calculating the KL divergence of $f(\mathbf{s}|\hat{\mathbf{s}})$ from $f(\mathbf{s}|\mathbf{s}_i)$. PARQO′ leverages the same robustness metric (i.e. Equation (1)) as in PAR^{2}QO.



(a) Reduction by τ-Approximate Cover

(b) Reduction by Cost Profile Similarity

Figure 4: Plan cost distribution of remaining plans after different reduction methods for template 17 in JOB. Initially, 48 candidate plans are obtained by Opt from 50 selectivity probes across 6 clusters, separated by vertical dashed lines. We manually set the remaining plan size to 5, with each plot presenting one of these plans. For similarity-based reduction, plans within the same group of according remaining plan are also shown (gray lines).

Workload generation setup. For CARVER, as mentioned in Section 4.1, if $\gamma_{k,j}$ is a querylet with a combination of multiple parameterized predicates, we generate parameters for each binding independently. For Uniform workload generation, certain JOB templates, including 25, 28-31, may encounter memory overflow when performing full-table joins. To mitigate this, we apply a random sampling approach as mentioned in Section 4.2. We initially sample 10% data from each table and incrementally increase the sampling rate if the number of unique parameter settings in $\mathsf{params}(\Gamma)$ is insufficient.

For Historical workload generation, parameter settings for each predicate are extracted from the original benchmark queries in JOB and then permuted across all predicates within each template Γ to generate $\mathsf{params}(\Gamma)$. The synthesized queries are then split into two distinct sets for training and testing.

For each workload generation method, we apply a *posteriors filter* (Section 4.2) after the initial synthesis to ensure only *valid* queries are included. Similar guarantees are also discussed in [14]. This filter removes queries that return zero rows (before final aggregation), ensuring all queries in the workload produce at least one row. Beyond practical considerations, there is a technical necessity for this filtering due to PostgreSQL's plan hinting mechanism [35]. Without hinting, if a query contains a subquery that produces zero rows, the optimizer may detect this early and terminate execution prematurely. Such premature termination can lead to misleading performance evaluations, as the observed speed may stem from early termination rather than actual plan efficiency. However, when a complete physical plan is hinted, PostgreSQL follows the prescribed join order and join strategies step-by-step, executing the full plan regardless of intermediate zero-row results. To prevent premature termination and ensure meaningful query execution, we manually enforce that all queries in the workload return at least one row. We observe that in certain templates, the one-component scenario is inefficient when this filter is applied. Even after synthesizing over 10,000 initial queries, the number of valid queries remains insufficient—for example, in templates 24 and 29. For these cases, CARVER switches to the two-component mixture model, as described in Example 1, using equal weights. And this mixture ensures non-empty results.

E ROBUSTNESS VERIFICATION ON DATA SHIFTS

The four representative queries are selected based on the following criteria. Template 4 shows the largest degradation for PAR^{2}QO ($H4$ in Figure 2). Template 14 represents a scenario where neither PAR^{2}QO nor Kepler shows significant speedup or regression. Template 17 is a case where PAR^{2}QO outperforms Kepler, while Template 19 is where Kepler significantly outperforms PAR^{2}QO. These templates also vary in complexity, ranging from 5 to 10-table joins and 3 to 12 parameter bindings, ensuring a diverse evaluation. The detailed template-level performance across different slicing methods and workloads is presented in Figures 5 to 7. For clarity, we plot a horizontal dashed line (red) in some figures as a marker for significant speedups or regressions exceeding 1.2×. First, in category-slicing, where data distribution varies the most, we use "category" as the base instance. Query performance fluctuates across instances, yet Figure 5 shows that plans selected by PAR^{2}QO maintain robust performance across templates and workloads. Although PAR^{2}QO exhibits some degradation over PostgreSQL in certain cases (e.g., $H4$ and $C17$) when tested on other instances, the largest regression

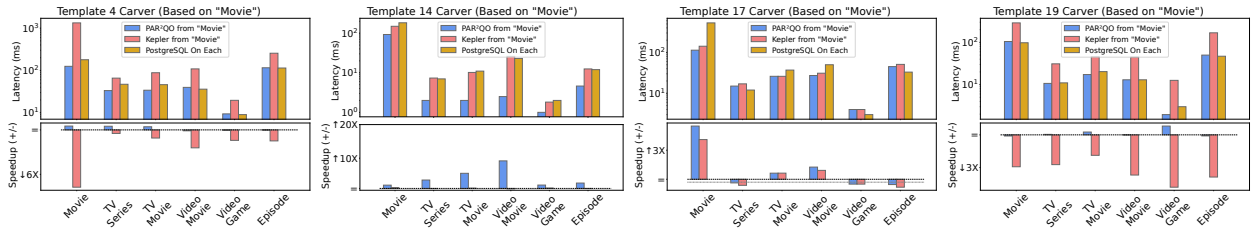
does not exceed −2×. Interestingly, some templates with modest initial speedups on the base instance perform significantly better on other instances, even surpassing PostgreSQL. Examples include $C14$, $U14$, $H14$, and $H19$. We do not intend to beat PostgreSQL's refreshed plan on these instances, since our current error model observed from the base instance is a static profile and does not explicitly account for possible data shifts. But the robustness consideration in PAR^{2}QO inherently promotes stability and prevents severe regressions. In contrast, Kepler suffers severe regressions in $C4$ and $C19$, with performance dropping reaching up to −6× compared to the baseline.

Second, in time-slicing, we use DB5 as the base instance. The speedups generally follow a unimodal distribution, peaking in middle instances and decreasing toward both extremes (e.g., $C19$). However, some templates deviate from this trend, such as $H17$ and $C17$. For example, the plan obtained for $H17$ from DB5 achieves a 3× speedup, but on DB1, which is farther from the base, the speedup increases to 7.15×. The only severe regression observed for PAR^{2}QO is $H19$ on DB1 (−2.68×), though it remains better than Kepler's −3.05×.

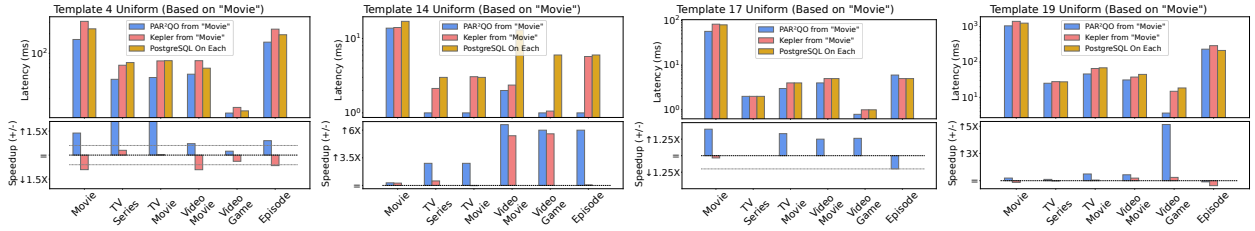
Finally, in random-slicing, we regard DB5 as the base instance. Most templates exhibit consistent execution performance across instances. PAR^{2}QO maintains both its speedups and low regression levels across different instances, demonstrating its robustness.

F WORKLOAD EFFECTIVENESS EVALUATION: AN ADJUSTMENT FOR KEPLER

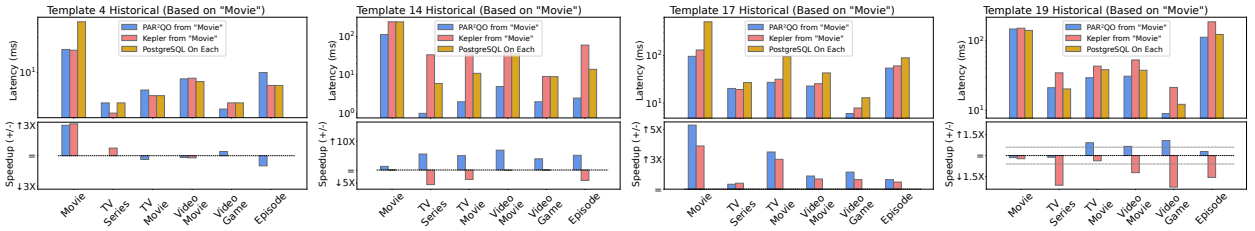
As discussed in Section 5.1, Kepler lacks the robustness protection of PAR^{2}QO, potentially leading to poor plan selection. To help ensure that our conclusion is not biased by outliers, we reexamine the results by excluding the single template causing the largest performance decline when trained on each workload. Note that this exclusion is based on performance decline, not execution time. After this adjustment, Kepler trained on CARVER demonstrates the best performance, reducing its latency to 432ms and achieving a speedup of 1.28×, surpassing Uniform (524ms) and Historical (508ms). These results underscore CARVER's effectiveness in generating workloads with comprehensive cardinality coverages: using CARVER as the training workload can enhance plan efficiency as well as robustness, and improve the ability to generalize to new workloads. These advantages position PAR^{2}QO as a valuable tool not only for evaluating PQO model performance but also for guiding its improvement.



(a) Category-slicing: Carver Workload

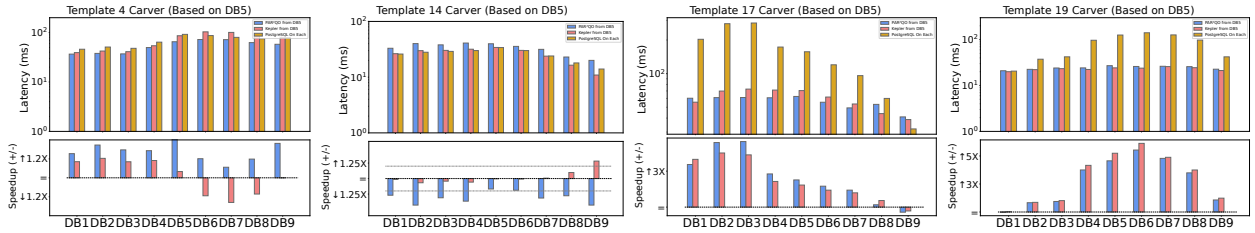


(b) Category-slicing: Uniform Workload

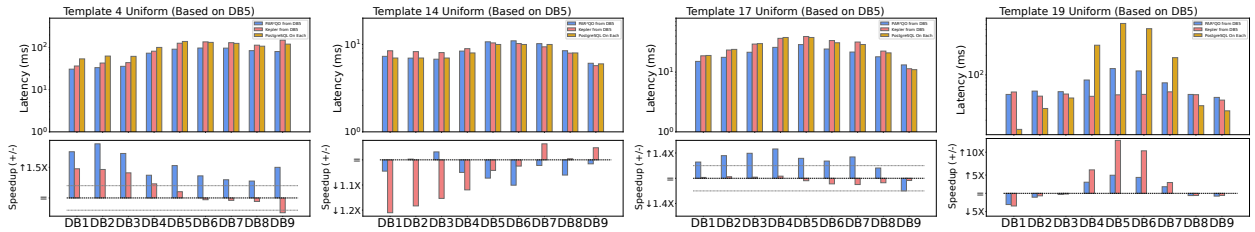


(c) Category-slicing: Historical Workload

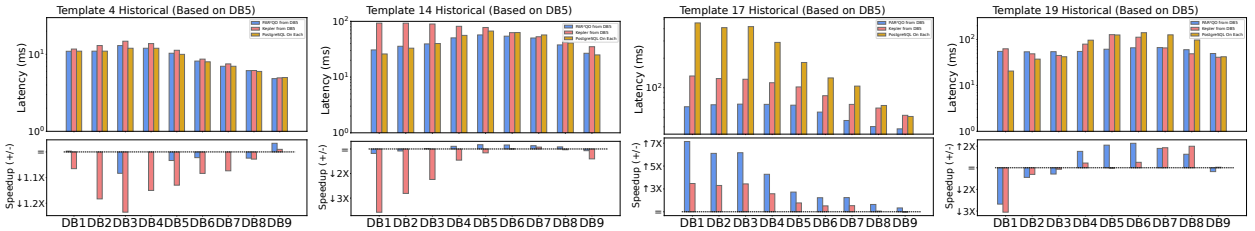
Figure 5: Robustness verification by category-slicing. "Movie" is the base database instance.



(a) Time-slicing: Carver Workload

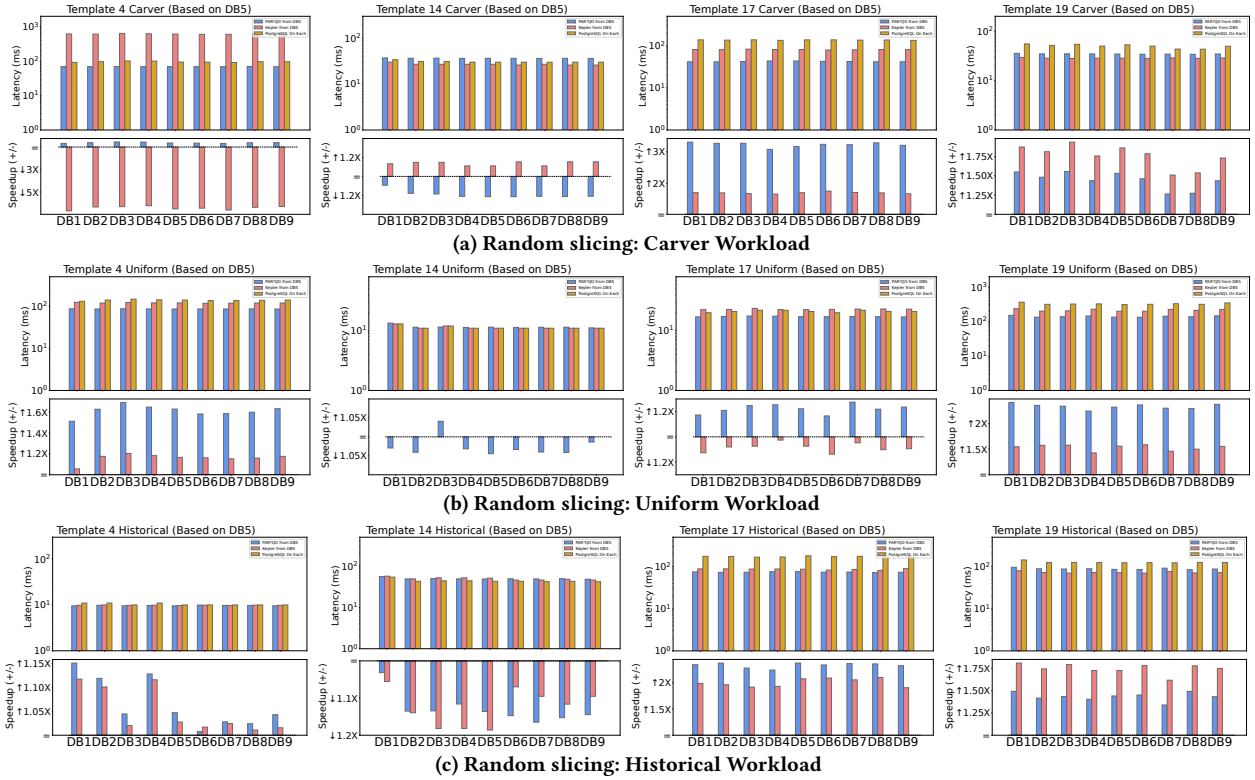


(b) Time-slicing: Uniform Workload



(c) Time-slicing: Historical Workload

Figure 6: Robustness verification by time-slicing. DB5 is the base database instance.



(a) Random slicing: Carver Workload

(b) Random slicing: Uniform Workload

(c) Random slicing: Historical Workload

Figure 7: Robustness verification by random-slicing. DB5 is the base database instance.